

Pillar 5: Capstone Project: Early Warning Student Dropout Risk Prediction System

Submitted by: Rex Villamar

24 September 2026

Selected Domain: Education (Predict Student Dropout Risk)

Dataset: *Predict Students' Dropout and Academic Success* (UCI Machine Learning Repository; Realinho et al., 2022), $N = 4,424$ students, 36 features

Target Environment: Google Colab / Jupyter Notebook

The Story in One Slide

Problem. Student dropout costs institutions tuition revenue and costs students their qualification. This notebook builds a classifier that flags, at the end of Semester 2, the students most likely to drop out, so advisors can act while later semesters remain.

Approach. Every design decision is made to serve one operating constraint: *missing a future dropout is far more costly than an unnecessary advisory check-in*, so the system is tuned and judged on recall (target ≥ 0.85), with precision tracked as the price.

Step	Question answered	What this notebook does
1	What are we predicting, and how is success judged?	Binary dropout target, decision point at end of Semester 2, recall target and F2 selection metric
2	What data do we have?	UCI dataset, quality audit, data dictionary
3	How does raw data become model input?	Cleaning audit, four engineered features, sensitive attributes withheld, RFECV selection, PCA diagnostics
4	Which model is best?	Four model families tuned by 5-fold CV on F2; selection on training CV only
5	Can the model be trusted and explained?	Leakage and overfitting audit, SHAP and LIME, fairness audit by group, evaluated bias mitigation, findings
6-7	How is it communicated and shipped?	Slide export for two audiences, repository structure

Reading guide. Each slide pairs the *reasoning* (markdown) with the *evidence* (code and outputs). Blocks titled *takeaways* are computed from the run itself, so every number quoted

in them matches the outputs above them.

Step 1: Problem Understanding & Framing

1.1 Business Context & Problem Statement

Higher education institutions face significant revenue loss and reduced institutional ranking due to unexpected student dropouts. Acquiring a new student costs significantly more than retaining an existing student. By detecting dropout indicators early (at the end of Semester 2, once both semesters' academic records are available), academic advisory teams can deliver targeted interventions—such as tutoring, counseling, or financial aid updates—before a student officially drops out.

1.2 Data Science Task Framing & Target Definition

- Task Classification: Supervised Binary Classification.
- Decision Point: End of Semester 2. Each student is scored once their 1st- and 2nd-semester records are complete, leaving later semesters in which an intervention can still change the outcome. Every feature used is available by this point.
- Feature Vector (X): Demographic background (excluding the sensitive attributes below), socio-economic factors, application metadata, and 1st/2nd-semester academic performance.
- Responsible-AI Constraint: **Gender** and **Marital status** are not model inputs; they are kept only to audit whether outcomes differ across groups.
- Target Mapping (y):

$$\text{Target } (y) = \begin{cases} 1 & \text{if Status= 'Dropout' (High Risk)} \\ 0 & \text{if Status} \in \{ \text{'Graduate', 'Enrolled'} \} \text{ (Retained)} \end{cases}$$

1.3 Asymmetric Cost Matrix & Evaluation Metrics

Actual / Predicted	Predicted High Risk ($\hat{y} = 1$)
Actual Dropout ($y = 1$)	True Positive (TP) — Successful Intervention
Actual Retained ($y = 0$)	False Positive (FP) — Low Cost (Advisory Check -

- Primary Technical Metric: Recall (Sensitivity) on Class 1 ($\frac{TP}{TP+FN}$), targeting $\geq 85\%$ detection rate for actual dropouts.

- Secondary Technical Metrics: F2-Score (emphasizing Recall twice as much as Precision), ROC-AUC, and PR-AUC.
 - Evaluation Protocol: Stratified 80/20 train/test split (`random_state=42`). Models are tuned and selected on training-set cross-validation (F_2); the held-out test set only *reports* final performance at a fixed decision threshold of 0.50. Deployment criterion: $\text{Recall} \geq 85\%$, with Precision tracked as the cost of that recall.
 - Business KPIs:
 1. Student Retention Rate Increase (%): Year-over-year improvement in retained students.
 2. Net Saved Tuition Revenue (\$):

$$(\text{Retained Students} \times \text{Annual Tuition}) - \text{Intervention Operating Cost}.$$
 - How the business KPIs are evaluated: the executive deck converts the measured recall and precision into students retained, tuition revenue preserved and advisory workload (Step 6).
-

Step 2: Data Collection & Understanding

- Applied Concept: Automated Ingestion Pipeline, Data Quality Auditing, and Feature Dictionary Profiling.
- Source and justification: *Predict Students' Dropout and Academic Success* (UCI repository; Realinho et al., 2022) links enrolment records with first- and second-semester academic results for $N = 4,424$ students and 36 features, and labels each student *Dropout*, *Enrolled* or *Graduate*. It is a public, documented, real institutional dataset whose columns match the decision we want to support (who is at risk after two semesters).
- How It Works:
 - Ingestion and quality audit: The zip is downloaded programmatically, parsed (; delimited) and column names are stripped of stray whitespace. Missing-value and duplicate counts are checked before any modelling.
 - Class ratio: Graduate 49.93%, Dropout 32.12%, Enrolled 17.95%. Collapsing *Graduate* and *Enrolled* into *Retained* (Step 3) yields a binary target with a 32% positive class.
 - Data dictionary: Every column is assigned to one of four domain groups plus the target, with its data type and range or cardinality:
 - *Demographic*: marital status, nationality, gender, age at enrolment, international, displaced, special educational needs.
 - *Socio-Economic*: parents' qualifications and occupations, debtor, scholarship holder, tuition fees up to date.

- *Application and Macroeconomic*: application mode and order, course, attendance mode, previous qualification and grade, admission grade, unemployment, inflation, GDP.
- *Academic (Semester 1 and 2)*: credited, enrolled, evaluated, approved units, grades and units without evaluation, per semester.
- Encoding caveat: categorical variables arrive integer-coded; their code meanings are in the UCI documentation, and the dictionary reports each one's cardinality.

```
In [1]: #Step 2: Data Collection, Profiling & Data Dictionary Generation
import pandas as pd
import numpy as np
import urllib.request
import zipfile
import os
from IPython.display import display, Markdown

# 1. Automated Dataset Download from UCI Repository
dataset_url = "https://archive.ics.uci.edu/static/public/697/predict+students+dropout+and+a
zip_path = "student_data.zip"
extract_path = "data/raw"

os.makedirs(extract_path, exist_ok=True)
if not os.path.exists("data/raw/data.csv"):
    urllib.request.urlretrieve(dataset_url, zip_path)
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_path)

# Load raw dataset (UCI data uses ';' delimiter) and strip stray whitespace from column nam
raw_df = pd.read_csv(os.path.join(extract_path, "data.csv"), sep=';')
raw_df.columns = raw_df.columns.str.strip()

# 2. Dataset Overview & Quality Audit
class_share = raw_df['Target'].value_counts(normalize=True)
print("=" * 60)
print(f"Dataset Shape: {raw_df.shape[0]} Rows, {raw_df.shape[1]} Columns (36 features + Tar
print(f"Missing Values Count: {raw_df.isnull().sum().sum()}")
print(f"Duplicate Rows Count: {raw_df.duplicated().sum()}")
print("Target Class Distribution:\n", class_share)
print("=" * 60)

# 3. Data Dictionary (domain category, type, range / cardinality)
demographic = ['Marital status', 'Nationality', 'Gender', 'Age at enrollment', 'Internation
socio_economic = ["Mother's qualification", "Father's qualification", "Mother's occupation"
                  'Scholarship holder', 'Debtor', 'Tuition fees up to date']
application_macro = ['Application mode', 'Application order', 'Course', 'Daytime/evening at
                    'Previous qualification (grade)', 'Admission grade', 'Inflation rate',

def categorise(col):
    if col == 'Target':
        return 'Target'
    if col in demographic:
        return 'Demographic'
    if col in socio_economic:
```

```

        return 'Socio-Economic'
    if col in application_macro:
        return 'Application/Macro'
    return 'Academic (Sem 1 & 2)'

data_dict = pd.DataFrame({
    'Feature Name': raw_df.columns,
    'Category': [categorise(c) for c in raw_df.columns],
    'Data Type': [str(dtype) for dtype in raw_df.dtypes],
    'Min Value / Unique': [raw_df[c].min() if pd.api.types.is_numeric_dtype(raw_df[c]) else
    'Max Value / Example': [raw_df[c].max() if pd.api.types.is_numeric_dtype(raw_df[c]) else
    })

print("\n--- DATA DICTIONARY (all columns) ---")
display(data_dict)

# 4. Takeaways computed from the data
cat_counts = data_dict['Category'].value_counts()
display(Markdown(f"""
**Step 2 takeaways (computed from the data)**

- **Size and quality:** {raw_df.shape[0]:,} students, {raw_df.shape[1] - 1} features plus t
- **Class balance:** Graduate {class_share['Graduate']:.1%}, Dropout {class_share['Dropout']
- **Feature groups:** {'', '.join(f'{k}: {v}' for k, v in cat_counts.items() if k != 'Target
- **Encoding:** categorical variables (course, application mode, qualifications, occupation
"""))

```

```

=====
Dataset Shape: 4424 Rows, 37 Columns (36 features + Target)
Missing Values Count: 0
Duplicate Rows Count: 0
Target Class Distribution:
  Target
Graduate    0.499322
Dropout     0.321203
Enrolled    0.179476
Name: proportion, dtype: float64
=====

```

```

--- DATA DICTIONARY (all columns) ---

```

	Feature Name	Category	Data Type	Min Value / Unique	Max Value / Example
0	Marital status	Demographic	int64	1.00	6
1	Application mode	Application/Macro	int64	1.00	57
2	Application order	Application/Macro	int64	0.00	9
3	Course	Application/Macro	int64	33.00	9991
4	Daytime/evening attendance	Application/Macro	int64	0.00	1
5	Previous qualification	Application/Macro	int64	1.00	43
6	Previous qualification (grade)	Application/Macro	float64	95.00	190.0
7	Nacionality	Demographic	int64	1.00	109
8	Mother's qualification	Socio-Economic	int64	1.00	44
9	Father's qualification	Socio-Economic	int64	1.00	44
10	Mother's occupation	Socio-Economic	int64	0.00	194
11	Father's occupation	Socio-Economic	int64	0.00	195
12	Admission grade	Application/Macro	float64	95.00	190.0
13	Displaced	Demographic	int64	0.00	1
14	Educational special needs	Demographic	int64	0.00	1
15	Debtor	Socio-Economic	int64	0.00	1
16	Tuition fees up to date	Socio-Economic	int64	0.00	1
17	Gender	Demographic	int64	0.00	1
18	Scholarship holder	Socio-Economic	int64	0.00	1
19	Age at enrollment	Demographic	int64	17.00	70
20	International	Demographic	int64	0.00	1
21	Curricular units 1st sem (credited)	Academic (Sem 1 & 2)	int64	0.00	20
22	Curricular units 1st sem (enrolled)	Academic (Sem 1 & 2)	int64	0.00	26
23	Curricular units 1st sem (evaluations)	Academic (Sem 1 & 2)	int64	0.00	45
24	Curricular units 1st sem (approved)	Academic (Sem 1 & 2)	int64	0.00	26
25	Curricular units 1st sem (grade)	Academic (Sem 1 & 2)	float64	0.00	18.875

	Feature Name	Category	Data Type	Min Value / Unique	Max Value / Example
26	Curricular units 1st sem (without evaluations)	Academic (Sem 1 & 2)	int64	0.00	12
27	Curricular units 2nd sem (credited)	Academic (Sem 1 & 2)	int64	0.00	19
28	Curricular units 2nd sem (enrolled)	Academic (Sem 1 & 2)	int64	0.00	23
29	Curricular units 2nd sem (evaluations)	Academic (Sem 1 & 2)	int64	0.00	33
30	Curricular units 2nd sem (approved)	Academic (Sem 1 & 2)	int64	0.00	20
31	Curricular units 2nd sem (grade)	Academic (Sem 1 & 2)	float64	0.00	18.571429
32	Curricular units 2nd sem (without evaluations)	Academic (Sem 1 & 2)	int64	0.00	12
33	Unemployment rate	Application/Macro	float64	7.60	16.2
34	Inflation rate	Application/Macro	float64	-0.80	3.7
35	GDP	Application/Macro	float64	-4.06	3.51
36	Target	Target	object	3.00	['Dropout' 'Graduate']

Step 2 takeaways (computed from the data)

- Size and quality: 4,424 students, 36 features plus the 3-class **Target** ; 0 missing values and 0 duplicate rows, so no imputation or de-duplication is required.
- Class balance: Graduate 49.9%, Dropout 32.1%, Enrolled 17.9%. Collapsing Graduate + Enrolled into *Retained* gives a binary problem with a 32.1% positive class, i.e. moderate imbalance that must be handled in modelling and in metric choice.
- Feature groups: Academic (Sem 1 & 2): 12, Application/Macro: 10, Demographic: 7, Socio-Economic: 7.
- Encoding: categorical variables (course, application mode, qualifications, occupations, nationality, etc.) are already integer-coded in the source file; the dictionary reports their cardinality, while code meanings are given in the UCI documentation.

Step 3: Data Preprocessing, EDA & Feature Engineering

- Applied Concept: Target Binarization, Data-Quality and Outlier Audit, Domain Feature Synthesis, Applied EDA, Sensitive-Attribute Policy, RFECV Selection, and PCA Diagnostics.
- How It Works:
 - Target mapping: $y = 1$ for *Dropout* and $y = 0$ for *Graduate* or *Enrolled*.
 - Cleaning audit: Nulls and duplicates are counted. Outliers are screened with the $1.5 \times \text{IQR}$ rule on grades, ages, unit counts and macro variables; flagged values are kept, because they are genuine observations, tree ensembles are insensitive to them, and Logistic Regression standardises its inputs inside its pipeline.
 - Domain features (computed row-wise, so they cannot leak across the split):
 - *Approval rate*: $\text{Approval_Rate}_s = \frac{\text{Units Approved}_s}{\text{Units Enrolled}_s + 10^{-5}}$ for each semester s .
 - *Academic momentum*: $\text{Grade_Trend} = \text{Grade}_{\text{Sem2}} - \text{Grade}_{\text{Sem1}}$.
 - *Financial stress*:
 $\text{Debtor} + (1 - \text{Tuition Up To Date}) + (1 - \text{Scholarship Holder})$.
 - Applied EDA: three views test whether these ideas carry signal: a boxplot of `Grade_Trend` by outcome, dropout counts by `Financial_Stress_Index`, and the ten features most correlated with the target.
 - Sensitive-attribute policy: `Gender` and `Marital status` are removed from the model inputs and kept in a separate audit table, used only in Step 5 to measure whether outcomes differ by group.
 - Split-then-fit protocol: a stratified 80/20 split (`random_state=42`) is made first; the `StandardScaler`, RFECV and PCA are then fitted on training data only, so no test information influences preprocessing.
 - Feature selection (wrapper): RFECV with a Random Forest and recall scoring removes features that do not help cross-validated recall; the selected set is what Step 4 trains on.
 - Dimensionality reduction: PCA retaining 95% of the variance is used to diagnose redundancy and to visualise class overlap in two dimensions. It is not fed to the models, because principal components would make SHAP explanations in Step 5 uninterpretable.

```
In [2]: #Step 3: Cleaning Audit, Feature Engineering, EDA, Selection & PCA
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, Markdown
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import RFECV
from sklearn.ensemble import RandomForestClassifier
from sklearn.decomposition import PCA

# 3.1 Target Binarization
df = raw_df.copy()
# Map 3-class target into Binary: 1 = Dropout, 0 = Retained (Graduate + Enrolled)
```

```

df['Target_Binary'] = df['Target'].apply(lambda x: 1 if x == 'Dropout' else 0)
df.drop(columns=['Target'], inplace=True)

# 3.2 Data-Quality Audit: nulls, duplicates, outliers (IQR rule)
n_nulls, n_dups = int(df.isnull().sum().sum()), int(df.duplicated().sum())
print(f"Missing values: {n_nulls} | Duplicate rows: {n_dups}")

outlier_cols = ['Previous qualification (grade)', 'Admission grade', 'Age at enrollment',
                'Curricular units 1st sem (grade)', 'Curricular units 2nd sem (grade)',
                'Curricular units 1st sem (enrolled)', 'Curricular units 2nd sem (enrolled)',
                'Curricular units 1st sem (approved)', 'Curricular units 2nd sem (approved)',
                'Unemployment rate', 'Inflation rate', 'GDP']

def iqr_outliers(s):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return int(((s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)).sum())

outlier_table = pd.DataFrame({
    'Feature': outlier_cols,
    'Outliers (IQR rule)': [iqr_outliers(df[c]) for c in outlier_cols],
})
outlier_table['% of rows'] = (100 * outlier_table['Outliers (IQR rule)'] / len(df)).round(2)
outlier_table = outlier_table.sort_values('Outliers (IQR rule)', ascending=False).reset_index()
print("\nOutlier screen (1.5 x IQR rule) - values are flagged, not removed:")
print(outlier_table.to_string(index=False))

# 3.3 Domain-Derived Feature Engineering (row-wise, so no train/test leakage)
df['Approval_Rate_Sem1'] = df['Curricular units 1st sem (approved)'] / (df['Curricular unit
df['Approval_Rate_Sem2'] = df['Curricular units 2nd sem (approved)'] / (df['Curricular unit
df['Grade_Trend'] = df['Curricular units 2nd sem (grade)'] - df['Curricular units 1st sem (
df['Financial_Stress_Index'] = df['Debtor'] + (1 - df['Tuition fees up to date']) + (1 - df
ENGINEERED = ['Approval_Rate_Sem1', 'Approval_Rate_Sem2', 'Grade_Trend', 'Financial_Stress_
print(f"\nNew Features Created: {' '.join(ENGINEERED)}")

# 3.4 Applied EDA
fig, axes = plt.subplots(1, 3, figsize=(20, 5))
sns.boxplot(x='Target_Binary', y='Grade_Trend', hue='Target_Binary', data=df, ax=axes[0], p
axes[0].set_title('Academic Momentum (Grade Trend) vs Dropout Risk')
axes[0].set_xticks([0, 1])
axes[0].set_xticklabels(['Retained (0)', 'Dropout (1)'])

sns.countplot(x='Financial_Stress_Index', hue='Target_Binary', data=df, ax=axes[1], palette
axes[1].set_title('Financial Stress Index vs Dropout Count')

corr_target = (df.drop(columns=['Target_Binary']).corrwith(df['Target_Binary'])
                .sort_values(key=lambda s: s.abs(), ascending=False).head(10))
sns.barplot(x=corr_target.values, y=corr_target.index, ax=axes[2], color='#3b82f6')
axes[2].set_title('Top 10 Features by |Correlation| with Dropout')
axes[2].set_xlabel('Pearson correlation with Target_Binary')
plt.tight_layout()
plt.subplots_adjust(wspace=0.5)
plt.show()

# 3.5 Sensitive Attributes: withheld from model inputs, retained ONLY for the fairness audi
SENSITIVE_COLS = ['Gender', 'Marital status']

```

```

X_full = df.drop(columns=['Target_Binary'])          # every column, used only to build aud
y = df['Target_Binary']
X = X_full.drop(columns=SENSITIVE_COLS)             # model inputs

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, s
audit_train = X_full.loc[X_train.index]
audit_test = X_full.loc[X_test.index]
print(f"\nSensitive attributes withheld from the model: {SENSITIVE_COLS}")
print(f"Train: {X_train.shape} | Test: {X_test.shape}")

# 3.6 Scaling, RFECV Feature Selection & PCA (all fitted on TRAINING data only)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

rfecv = RFECV(estimator=RandomForestClassifier(n_estimators=50, random_state=42), step=1, c
rfecv.fit(X_train_scaled, y_train)
selected_features = X_train.columns[rfecv.support_].tolist()
dropped_features = [c for c in X_train.columns if c not in selected_features]
n_before_selection = X_train.shape[1]
X_train = X_train[selected_features]                # downstream modelling uses the RFECV-selected fea
X_test = X_test[selected_features]
print(f"RFECV Selected Features: {len(selected_features)} out of {n_before_selection}")
if dropped_features:
    print("Dropped by RFECV:", dropped_features)

pca = PCA(n_components=0.95, random_state=42)
pca.fit(X_train_scaled)
n_pca = pca.n_components_
pca2 = PCA(n_components=2, random_state=42)
Z = pca2.fit_transform(X_train_scaled)
print(f"PCA: {n_pca} components retain 95% of the variance (of {n_before_selection} scaled

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
full_pca = PCA(random_state=42).fit(X_train_scaled)
axes[0].plot(np.arange(1, len(full_pca.explained_variance_ratio_) + 1), np.cumsum(full_pca.
axes[0].axhline(0.95, color='red', ls='--', label='95% variance')
axes[0].axvline(n_pca, color='gray', ls=':', label=f'{n_pca} components')
axes[0].set_xlabel('Number of principal components')
axes[0].set_ylabel('Cumulative explained variance')
axes[0].set_title('PCA: Cumulative Explained Variance')
axes[0].legend()
sc = axes[1].scatter(Z[:, 0], Z[:, 1], c=y_train.values, cmap='coolwarm', alpha=0.35, s=10)
axes[1].set_xlabel(f'PC1 ({pca2.explained_variance_ratio_[0]:.1%})')
axes[1].set_ylabel(f'PC2 ({pca2.explained_variance_ratio_[1]:.1%})')
axes[1].set_title('Training Students in the First Two Principal Components (red = dropout)')
plt.tight_layout()
plt.show()

# 3.7 Takeaways computed from the data
dropped_txt = ', '.join('\' ' + c + ' \' for c in dropped_features[:8]) + (f' and {len(dropped
gt_med = df.groupby('Target_Binary')['Grade_Trend'].median()
fsi_rate = (df.groupby('Financial_Stress_Index')['Target_Binary'].mean() * 100).round(1)
n_flag_cols = int((outlier_table['Outliers (IQR rule)'] > 0).sum())
display(Markdown(f"""
**Step 3 takeaways (computed from the data)**

```

```

- **Cleaning:** {n_nulls} missing values and {n_dups} duplicates, so nothing was imputed or
- **Split:** {len(X_train):,} training / {len(X_test):,} test students, stratified (dropout
- **Sensitive attributes:** 'Gender' and 'Marital status' are excluded from the model input
- **EDA:** median 'Grade_Trend' is {gt_med[0]:+.2f} for retained vs {gt_med[1]:+.2f} for dr
- **Feature selection:** RFECV (recall-scored) kept {len(selected_features)} of {n_before_s
- **Dimensionality reduction:** {n_pca} principal components are needed for 95% of the vari
"""
))

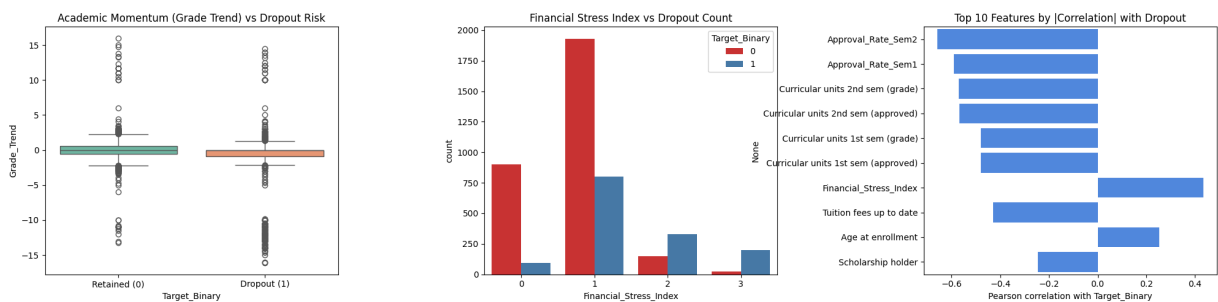
```

Missing values: 0 | Duplicate rows: 0

Outlier screen (1.5 x IQR rule) - values are flagged, not removed:

Feature	Outliers (IQR rule)	% of rows
Curricular units 2nd sem (grade)	877	19.82
Curricular units 1st sem (grade)	726	16.41
Age at enrollment	441	9.97
Curricular units 1st sem (enrolled)	424	9.58
Curricular units 2nd sem (enrolled)	369	8.34
Curricular units 1st sem (approved)	180	4.07
Previous qualification (grade)	179	4.05
Admission grade	86	1.94
Curricular units 2nd sem (approved)	44	0.99
Unemployment rate	0	0.00
Inflation rate	0	0.00
GDP	0	0.00

New Features Created: Approval_Rate_Sem1, Approval_Rate_Sem2, Grade_Trend, Financial_Stress_Index

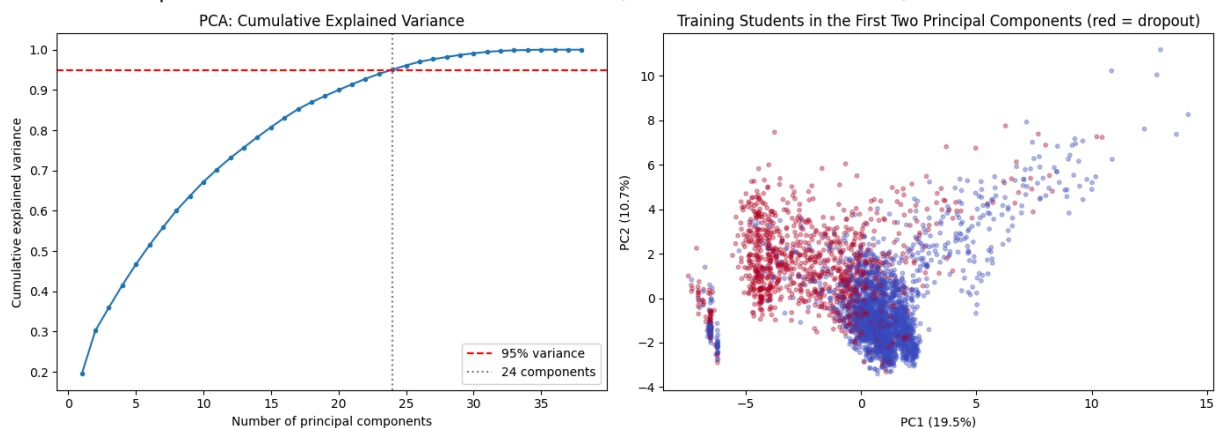


Sensitive attributes withheld from the model: ['Gender', 'Marital status']

Train: (3539, 38) | Test: (885, 38)

RFECV Selected Features: 38 out of 38

PCA: 24 components retain 95% of the variance (of 38 scaled features)



Step 3 takeaways (computed from the data)

- Cleaning: 0 missing values and 0 duplicates, so nothing was imputed or dropped. 9 of 12 continuous / count features contain IQR-rule outliers (largest: `Curricular units 2nd sem (grade)` , 19.82% of rows). These are genuine observations (e.g. mature students, many credited units), so they are retained; the tree ensembles are insensitive to them and Logistic Regression standardises its inputs inside its pipeline.
 - Split: 3,539 training / 885 test students, stratified (dropout rate 32.1% train, 32.1% test). Scaler, RFECV and PCA are fitted on training data only.
 - Sensitive attributes: `Gender` and `Marital status` are excluded from the model inputs and kept in `audit_train` / `audit_test` for the Step 5 fairness audit.
 - EDA: median `Grade_Trend` is +0.00 for retained vs +0.00 for dropout students; dropout rate by `Financial_Stress_Index` level is 0: 9.3%, 1: 29.3%, 2: 69.2%, 3: 88.8%.
 - Feature selection: RFECV (recall-scored) kept 38 of 38 features (it dropped none, so every feature contributed to cross-validated recall). The selected set is what Step 4 trains on.
 - Dimensionality reduction: 24 principal components are needed for 95% of the variance of 38 features, so the information is spread across many directions rather than a few; the PC1-PC2 projection shows the classes overlapping heavily, i.e. no clean linear separation. PCA is therefore used for diagnosis and visualisation only; models keep the original features so SHAP explanations stay interpretable.
-

Step 4: Model Implementation & Comparison

- Applied Concept: Multi-Model Implementation, Class-Imbalance Handling, Stratified Cross-Validation, Hyperparameter Optimisation, Leakage-Free Model Selection, and Artefact Serialisation.
- How It Works:
 - Model suite (increasing capacity):
 - *Logistic Regression*: linear baseline; a `StandardScaler` sits inside its pipeline so scaling is re-fitted on every CV training fold.
 - *Random Forest*: bagged decision trees that reduce variance.
 - *XGBoost and LightGBM*: gradient-boosted trees that sequentially correct earlier errors; they can capture non-linear interactions such as approval rate combined with financial stress.
 - Class asymmetry: with only 32% dropouts, grids include class weighting (`class_weight` for scikit-learn and LightGBM, `scale_pos_weight` for XGBoost) so the minority class counts more in the loss.
 - Tuning objective: $F_2 = 5PR/(4P + R)$, i.e. F_β with $\beta = 2$, which weights recall twice as heavily as precision to mirror the cost matrix of Step 1.
 - Selection without leakage: each model is tuned by `GridSearchCV` with stratified 5-fold CV on the training set, and models are ranked by cross-validated F_2 . The hold-

out set is used only to *report* performance at a fixed threshold of 0.50; it plays no part in choosing the model.

- Reported metrics: recall, precision, F_2 , ROC-AUC and PR-AUC. The fold standard deviation of the winner is shown so the ranking margin can be judged against noise.
- Reproducibility: every tuned model (`.joblib`), the comparison table (`.csv`) and the final configuration (`.json` : parameters, threshold, seed, split, feature list, excluded attributes) are written to `models/`.

```
In [3]: # Step 4: Multi-Model Training, Tuning, Evaluation & Serialization
import os
import json
import joblib
import pandas as pd
import numpy as np
from IPython.display import display, Markdown

# Machine Learning & Metrics
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import make_scorer, fbeta_score, recall_score, precision_score, roc_auc_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

assert 'X_train' in globals(), "Run Step 3 first (X_train / X_test / y_train / y_test are c

# 0. Setup Directory
os.makedirs('models', exist_ok=True)
print("Directory 'models/' ready for saving model artifacts.\n")

# 1. Custom Metric Definition (F2-Score): weighs Recall twice as heavily as Precision
f2_scorer = make_scorer(fbeta_score, beta=2)

# 2. Classifiers and Hyperparameter Grids
models_and_grids = {
    'Logistic Regression': {
        # Scaling lives inside the pipeline so it is re-fitted on each CV training fold (no
        'model': Pipeline([('scaler', StandardScaler()), ('clf', LogisticRegression(random_
        'param_grid': {
            'clf__C': [0.01, 0.1, 1, 10],
            'clf__class_weight': ['balanced', None],
            'clf__solver': ['liblinear', 'lbfgs']
        }
    },
    'Random Forest': {
        'model': RandomForestClassifier(random_state=42),
        'param_grid': {
            'n_estimators': [100, 200],
            'max_depth': [5, 10, None],
            'class_weight': ['balanced', 'balanced_subsample', None],
            'min_samples_split': [2, 5]
```

```

    },
    'XGBoost': {
        'model': XGBClassifier(random_state=42, eval_metric='logloss'),
        'param_grid': {
            'n_estimators': [100, 200],
            'max_depth': [3, 5, 7],
            'learning_rate': [0.01, 0.1, 0.2],
            'scale_pos_weight': [1, 3, 5] # Boosts recall on imbalanced data
        }
    },
    'LightGBM': {
        'model': LGBMClassifier(random_state=42, verbose=-1),
        'param_grid': {
            'n_estimators': [100, 200],
            'max_depth': [3, 5, 7],
            'learning_rate': [0.01, 0.1, 0.2],
            'class_weight': ['balanced', None]
        }
    }
}

# 3. Model Tuning and Cross-Validation (selection uses TRAINING-set CV only)
cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results_list = []
trained_models = {}

print("Starting Hyperparameter Tuning (Optimizing for F2-Score)...\n")

for model_name, config in models_and_grids.items():
    print(f"--- Tuning {model_name} ---")
    grid_search = GridSearchCV(
        estimator=config['model'],
        param_grid=config['param_grid'],
        scoring=f2_scorer,
        cv=cv_strategy,
        n_jobs=-1,
        verbose=0
    )
    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_
    trained_models[model_name] = best_model

# Hold-out evaluation (reported, NOT used for selection)
y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)[:, 1]

results_list.append({
    'Model': model_name,
    'Best CV F2-Score': grid_search.best_score_,
    'CV F2 Std': grid_search.cv_results_['std_test_score'][grid_search.best_index_],
    'Test Recall': recall_score(y_test, y_pred),
    'Test Precision': precision_score(y_test, y_pred, zero_division=0),
    'Test F2-Score': fbeta_score(y_test, y_pred, beta=2),
    'Test ROC-AUC': roc_auc_score(y_test, y_proba),
    'Test PR-AUC': average_precision_score(y_test, y_proba),
})

```

```

        'Best Parameters': grid_search.best_params_
    })
    print(f"Done! Best CV F2-Score: {grid_search.best_score_:.4f} | Test F2-Score: {results

# 4. Tabular Summary, ranked by cross-validated F2 (model selection must not peek at the te
summary_df = pd.DataFrame(results_list).sort_values(by='Best CV F2-Score', ascending=False)

print("=" * 100)
print("                                MODEL PERFORMANCE COMPARISON TABLE (ranked by CV F2)")
print("=" * 100)
display_cols = ['Model', 'Best CV F2-Score', 'CV F2 Std', 'Test Recall', 'Test Precision',
print(summary_df[display_cols].to_string(index=False, float_format=lambda x: f"{x:.4f}"))

# 5. Save Artifacts: every tuned model, the comparison table, and the final configuration
best_row = summary_df.iloc[0]
best_model_name = best_row['Model']
best_model_obj = trained_models[best_model_name]

for name, mdl in trained_models.items():
    joblib.dump(mdl, f"models/{name.lower().replace(' ', '_')}.joblib")
model_path = f"models/best_model_{best_model_name.lower().replace(' ', '_')}.joblib"
joblib.dump(best_model_obj, model_path)
summary_df.to_csv('models/model_comparison.csv', index=False)
with open('models/best_model_config.json', 'w') as f:
    json.dump({
        'best_model': best_model_name,
        'best_parameters': best_row['Best Parameters'],
        'selection_metric': 'mean 5-fold CV F2 on the training set',
        'decision_threshold': 0.5,
        'random_state': 42,
        'test_size': 0.2,
        'excluded_sensitive_attributes': SENSITIVE_COLS,
        'features': X_train.columns.tolist(),
    }, f, indent=2, default=str)

print("\n" + "=" * 100)
print(f" Selected Model (highest CV F2): {best_model_name}")
print(f" Artifacts saved: {model_path}, models/model_comparison.csv, models/best_model_conf
print("=" * 100)

# 6. Takeaways computed from the results
second = summary_df.iloc[1]
cv_gap = best_row['Best CV F2-Score'] - second['Best CV F2-Score']
distinct = cv_gap > best_row['CV F2 Std']
display(Markdown(f"""
**Step 4 takeaways (computed from the results)**

- **Selected model:** {best_model_name}, ranked first on cross-validated F2 ({best_row[
- **Hold-out performance:** recall {best_row['Test Recall']:.3f}, precision {best_row['Test
- **Margin over the runner-up ({second['Model']}):** {cv_gap:.4f} CV-F2, which is 'larger'
- **Reproducibility:** all tuned models, the comparison table and the final configuration (
"""))

```

Directory 'models/' ready for saving model artifacts.

Starting Hyperparameter Tuning (Optimizing for F2-Score)...

--- Tuning Logistic Regression ---

Done! Best CV F2-Score: 0.8056 | Test F2-Score: 0.8229

--- Tuning Random Forest ---

Done! Best CV F2-Score: 0.7928 | Test F2-Score: 0.8241

--- Tuning XGBoost ---

Done! Best CV F2-Score: 0.8247 | Test F2-Score: 0.8554

--- Tuning LightGBM ---

Done! Best CV F2-Score: 0.8026 | Test F2-Score: 0.8374

=====

MODEL PERFORMANCE COMPARISON TABLE (ranked by CV F2)

=====

	Model	Best CV F2-Score	CV F2 Std	Test Recall	Test Precision	Test F2-Score
Test ROC-AUC	Test PR-AUC					
0.9374	XGBoost	0.8247	0.0199	0.9120	0.6852	0.8554
0.9278	0.9113					
	Logistic Regression	0.8056	0.0233	0.8345	0.7796	0.8229
0.9278	0.8982					
	LightGBM	0.8026	0.0148	0.8521	0.7832	0.8374
0.9354	0.9102					
	Random Forest	0.7928	0.0213	0.8345	0.7848	0.8241
0.9292	0.8965					

=====

Selected Model (highest CV F2): XGBoost

Artifacts saved: models/best_model_xgboost.joblib, models/model_comparison.csv, models/best_model_config.json

=====

Step 4 takeaways (computed from the results)

- Selected model: XGBoost, ranked first on cross-validated F2 (0.8247 +/- 0.0199); the test set played no part in the choice.
- Hold-out performance: recall 0.912, precision 0.685, F2 0.855, ROC-AUC 0.937, PR-AUC 0.911. The recall target of 0.85 is met, and the precision of 0.685 shows the advisory workload that recall costs (the rest of the flags are check-ins that turn out to be unnecessary).
- Margin over the runner-up (Logistic Regression): 0.0191 CV-F2, which is smaller than one fold standard deviation (0.0199), so the ranking is not statistically distinguishable; the choice among the top models is a judgement call.
- Reproducibility: all tuned models, the comparison table and the final configuration (parameters, threshold, seed, feature list) are saved in `models/`.

Step 5a: Is the Model Strong, Leakage-Free and Generalising?

- Applied Concept: Final Hold-Out Evaluation, Feature-Timing and Leakage Audit, Overfitting and Imbalance Diagnostics.
- Audit target: the model selected in Step 4, on the same split, at the same threshold (0.50), so every number here is directly comparable with the Step 4 table.
- How It Works:
 - Final performance: confusion matrix and precision-recall curve show the operating point against the recall target of 0.85 defined in Step 1.
 - Feature timing: the system decides at the end of Semester 2, so Semester 1 and 2 records are legitimate inputs. A timing table confirms every feature exists by then, and a with/without-Semester-2 comparison quantifies what the second semester adds.
 - Leakage screen: each feature's standalone ROC-AUC is computed ($\max(\text{AUC}, 1 - \text{AUC})$); a lone feature above 0.90 would signal that it leaks the outcome.
 - Overfitting: train versus validation scores from stratified 5-fold CV give the generalisation gap; a gap above 0.10 in F_2 is treated as overfitting and the conclusion text changes accordingly.
 - Imbalance: the cost of class weighting appears as lower precision, and is stated rather than hidden.

```
In [4]: # Step 5a: Final Performance, Leakage / Timing Audit & Overfitting Check
# DECISION POINT: END OF SEMESTER 2. This step audits the model SELECTED in Step 4 on the S
import sys
import subprocess
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, Markdown
warnings.filterwarnings('ignore')

for pkg in ['shap', 'lime']:
    try:
        __import__(pkg)
    except ImportError:
        subprocess.check_call([sys.executable, "-m", "pip", "install", pkg, "-q"])

from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold, cross_validate, cross_val_predict
from sklearn.metrics import (recall_score, precision_score, fbeta_score, roc_auc_score,
                             average_precision_score, confusion_matrix, make_scorer)
```

```

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

required = ['best_model_obj', 'best_model_name', 'X_train', 'X_test', 'y_train', 'y_test',
            'audit_train', 'audit_test', 'SENSITIVE_COLS', 'summary_df']
missing = [name for name in required if name not in globals()]
if missing:
    raise RuntimeError(f"Run Steps 2-4 first. Missing objects: {missing}")

model = best_model_obj                # model selected in Step 4
THRESHOLD = 0.5                       # decision threshold used for every reported metric
TARGET_RECALL = 0.85                  # deployment criterion defined in Step 1
y_test_arr, y_train_arr = y_test.values, y_train.values
y_prob = model.predict_proba(X_test)[: , 1]
y_pred = (y_prob >= THRESHOLD).astype(int)

# 1. FINAL HOLD-OUT PERFORMANCE
tn, fp, fn, tp = confusion_matrix(y_test_arr, y_pred, labels=[0, 1]).ravel()
base_metrics = {
    'Recall': recall_score(y_test_arr, y_pred),
    'Precision': precision_score(y_test_arr, y_pred, zero_division=0),
    'F2-Score': fbeta_score(y_test_arr, y_pred, beta=2),
    'ROC-AUC': roc_auc_score(y_test_arr, y_prob),
    'PR-AUC': average_precision_score(y_test_arr, y_prob),
}
print("=" * 80)
print(f"FINAL HOLD-OUT PERFORMANCE | {best_model_name} | threshold = {THRESHOLD} | decision")
print("=" * 80)
for k, v in base_metrics.items():
    print(f" {k:<10}: {v:.4f}")
print(f" Confusion matrix -> TP={tp} FN={fn} FP={fp} TN={tn}")
print(f" Target check (Recall >= {TARGET_RECALL}): {'PASSED' if base_metrics['Recall'] >=

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
sns.heatmap([[tn, fp], [fn, tp]], annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[0],
            xticklabels=['Pred. Retained', 'Pred. Dropout'], yticklabels=['Actual Retained'])
axes[0].set_title('Confusion Matrix (hold-out set)')
from sklearn.metrics import precision_recall_curve
prec_c, rec_c, _ = precision_recall_curve(y_test_arr, y_prob)
axes[1].plot(rec_c, prec_c, color='#3b82f6')
axes[1].scatter([base_metrics['Recall']], [base_metrics['Precision']], color='red', zorder=
axes[1].axvline(TARGET_RECALL, color='gray', ls='--', label=f'Recall target {TARGET_RECALL}')
axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curve')
axes[1].legend()
plt.tight_layout()
plt.show()

# 2. FEATURE TIMING & LEAKAGE AUDIT
print("\n" + "=" * 80)
print("LIMITATIONS AUDIT 1: FEATURE TIMING & LEAKAGE")
print("=" * 80)
sem2_cols = [c for c in X_train.columns if '2nd sem' in c or c in ('Approval_Rate_Sem2', 'G
sem1_cols = [c for c in X_train.columns if '1st sem' in c or c == 'Approval_Rate_Sem1']
pre_cols = [c for c in X_train.columns if c not in sem1_cols + sem2_cols]

```

```

timing_table = pd.DataFrame({
    'Feature group': ['Pre-enrolment / background', 'Semester 1 academic record', 'Semester 2 academic record'],
    'Available by': ['Enrolment', 'End of Semester 1', 'End of Semester 2 (decision point)'],
    '# features used': [len(pre_cols), len(sem1_cols), len(sem2_cols)],
})
print(timing_table.to_string(index=False))

m_no_sem2 = clone(model).fit(X_train[pre_cols + sem1_cols], y_train)
p_no_sem2 = m_no_sem2.predict_proba(X_test[pre_cols + sem1_cols])[:, 1]
auc_no_sem2 = roc_auc_score(y_test_arr, p_no_sem2)
rec_no_sem2 = recall_score(y_test_arr, (p_no_sem2 >= THRESHOLD).astype(int))
print(f"\nSame model WITHOUT Semester 2 data -> ROC-AUC {auc_no_sem2:.4f} | Recall {rec_no_sem2:.4f}")
print(f"Selected model WITH Semester 2 data -> ROC-AUC {base_metrics['ROC-AUC']:.4f} | Recall {rec_no_sem2:.4f}")

single_auc = pd.Series({c: roc_auc_score(y_train_arr, X_train[c]) for c in X_train.columns}).apply(lambda a: max(a, 1 - a)).sort_values(ascending=False)
print("\nTop 5 single-feature ROC-AUC values (training data) - leakage screen:")
print(single_auc.head(5).round(4).to_string())

# 3. OVERFITTING & CLASS IMBALANCE
print("\n" + "=" * 80)
print("LIMITATIONS AUDIT 2: OVERFITTING & CLASS IMBALANCE")
print("=" * 80)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = {'recall': 'recall', 'precision': 'precision', 'f2': make_scorer(fbeta_score, beta=2)}
cv_res = cross_validate(clone(model), X_train, y_train, cv=cv, scoring=scoring, return_train_score=True)
print("Stratified 5-Fold CV on training data (mean +/- std):")
for m in ['recall', 'precision', 'f2', 'roc_auc']:
    tr, va = cv_res[f'train_{m}'], cv_res[f'test_{m}']
    print(f" {m[:<10]} train = {tr.mean():.4f} +/- {tr.std():.4f} | validation = {va.mean():.4f} +/- {va.std():.4f}")
f2_gap = cv_res['train_f2'].mean() - cv_res['test_f2'].mean()
try:
    print("Selected parameters:", summary_df.loc[summary_df['Model'] == best_model_name, 'Best parameters'])
except Exception:
    pass

# 4. Takeaways computed from the results
display(Markdown(f"""
**Step 5a takeaways (computed from the results)**

- **Hold-out performance ({best_model_name}, threshold {THRESHOLD}):** recall {base_metrics['recall']:.4f}, precision {base_metrics['precision']:.4f}, f2 {base_metrics['f2']:.4f}
- **Operating point:** the model flags {int(y_pred.sum())} of {len(y_test_arr)} test students
- **Value of Semester 2 data:** ROC-AUC rises from {auc_no_sem2:.3f} (Semester 1 only) to {auc_with_sem2:.3f} (Semester 1 + 2)
- **Leakage screen:** the strongest single feature is '{single_auc.index[0]}' with AUC {single_auc[0]:.4f}
- **Generalisation:** train-minus-validation F2 gap is {f2_gap:.3f}, so the model {'**overfitted' if f2_gap > 0.05 else '**generalises well' if f2_gap < 0.05}
- **Class imbalance:** dropouts are {y_train_arr.mean():.1%} of training rows; class weight {1/y_train_arr.mean():.1f}

"""))

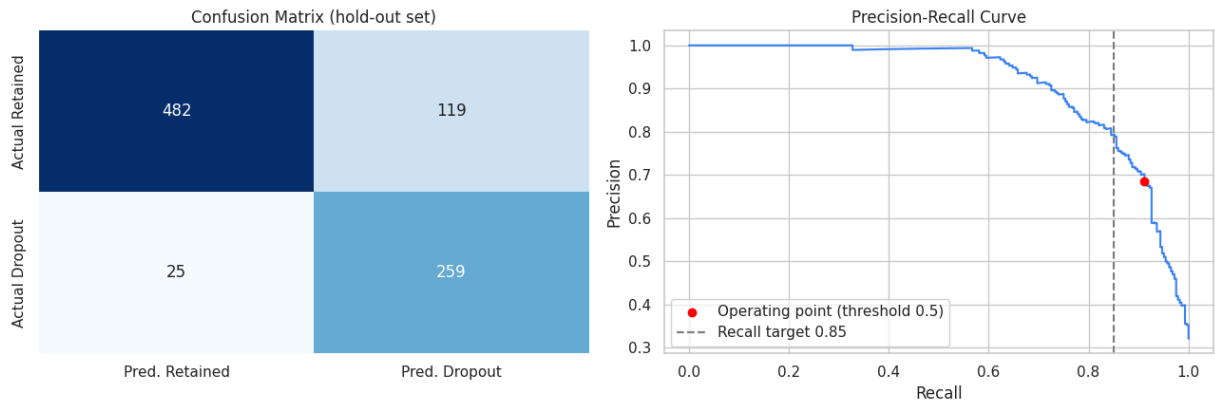
```

=====

FINAL HOLD-OUT PERFORMANCE | XGBoost | threshold = 0.5 | decision point: end of Semester 2

=====

Recall : 0.9120
 Precision : 0.6852
 F2-Score : 0.8554
 ROC-AUC : 0.9374
 PR-AUC : 0.9113
 Confusion matrix -> TP=259 FN=25 FP=119 TN=482
 Target check (Recall >= 0.85): PASSED



=====

LIMITATIONS AUDIT 1: FEATURE TIMING & LEAKAGE

=====

	Feature group	Available by	# feature
s used			
23	Pre-enrolment / background	Enrolment	
7	Semester 1 academic record	End of Semester 1	
8	Semester 2 academic record (incl. Grade_Trend)	End of Semester 2 (decision point)	

Same model WITHOUT Semester 2 data -> ROC-AUC 0.9118 | Recall 0.8803
 Selected model WITH Semester 2 data -> ROC-AUC 0.9374 | Recall 0.9120

Top 5 single-feature ROC-AUC values (training data) - leakage screen:

Approval_Rate_Sem2	0.8706
Curricular units 2nd sem (approved)	0.8528
Approval_Rate_Sem1	0.8359
Curricular units 1st sem (approved)	0.8157
Curricular units 2nd sem (grade)	0.8046

=====

LIMITATIONS AUDIT 2: OVERFITTING & CLASS IMBALANCE

=====

Stratified 5-Fold CV on training data (mean +/- std):

recall	train = 0.9323 +/- 0.0033	validation = 0.8795 +/- 0.0218	gap = +0.0528
precision	train = 0.6989 +/- 0.0059	validation = 0.6614 +/- 0.0306	gap = +0.0376
f2	train = 0.8739 +/- 0.0032	validation = 0.8247 +/- 0.0199	gap = +0.0492
roc_auc	train = 0.9590 +/- 0.0020	validation = 0.9228 +/- 0.0108	gap = +0.0362

Selected parameters: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 100, 'scale_pos_weight': 5}

Step 5a takeaways (computed from the results)

- Hold-out performance (XGBoost, threshold 0.5): recall 0.912, precision 0.685, F2 0.855, ROC-AUC 0.937, PR-AUC 0.911. The recall target of 0.85 is met.
- Operating point: the model flags 378 of 885 test students (42.7%); 119 of those flags are false alarms (one advisory check-in each) and 25 actual dropouts are missed.
- Value of Semester 2 data: ROC-AUC rises from 0.912 (Semester 1 only) to 0.937, and recall from 0.880 to 0.912, which is the gain bought by waiting one more semester.
- Leakage screen: the strongest single feature is `Approval_Rate_Sem2` with AUC 0.871; no single feature exceeds 0.90, so there is no evidence that any one variable leaks the outcome.
- Generalisation: train-minus-validation F2 gap is 0.049, so the model shows an acceptable generalisation gap (≤ 0.10).
- Class imbalance: dropouts are 32.1% of training rows; class weighting was part of every grid search, and the cost is the precision reported above.

Step 5b: Why Does the Model Flag a Student? (SHAP & LIME)

- Applied Concept: Global and Local Explainability.
- How It Works:
 - SHAP (global): `TreeExplainer` decomposes each prediction additively, $f(x) = \phi_0 + \sum_j \phi_j(x)$, in log-odds of dropout. The beeswarm plot shows each feature's effect for every student, and mean $|\phi_j|$ ranks the drivers.
 - LIME (local): a sparse linear surrogate is fitted around one correctly flagged dropout (median risk among true positives) to show which feature values moved that single prediction.
 - Why both: SHAP and LIME are independent attribution methods; agreement between them on the main drivers raises confidence that the explanation reflects the model.
 - Consistency check with Step 3: because gender and marital status were withheld, they cannot appear here; the drivers should instead be academic and financial signals.

```
In [5]: # Step 5b: Explainability - SHAP (global) and LIME (local)
import shap
import lime
import lime.lime_tabular

print("=" * 80)
print("EXPLAINABILITY: SHAP GLOBAL IMPORTANCE & LIME LOCAL EXPLANATION")
print("=" * 80)
X_shap = X_test
shap_units = 'log-odds of dropout'
try:
```

```

explainer_shap = shap.TreeExplainer(model) # exact and fast for tree ensembles
shap_values = explainer_shap.shap_values(X_shap)
if isinstance(shap_values, list):
    shap_dropout = shap_values[1]
elif getattr(shap_values, 'ndim', 2) == 3:
    shap_dropout = shap_values[:, :, 1]
else:
    shap_dropout = shap_values
except Exception:
    # Selected model is not a tree ensemble (e.g. the Logistic Regression pipeline): model-
    X_shap = X_test.sample(n=min(300, len(X_test)), random_state=42)
    background = shap.sample(X_train, 100, random_state=42)
    f_proba = lambda d: model.predict_proba(pd.DataFrame(d, columns=X_train.columns))[:, 1]
    shap_dropout = shap.Explainer(f_proba, background.values)(X_shap.values).values
    shap_units = 'probability of dropout'

print(f"[1/3] SHAP beeswarm: each dot is one student; SHAP values are in {shap_units} (n =
shap.summary_plot(shap_dropout, X_shap, show=False, max_display=15)
plt.title("SHAP Summary: Feature Impact on Dropout Risk (Selected Model)", fontsize=12, fon
plt.tight_layout()
plt.show()

print("[2/3] Mean |SHAP| ranking")
shap_rank = (pd.DataFrame({'Feature': X_shap.columns, 'Mean |SHAP|': np.abs(shap_dropout).m
    .sort_values('Mean |SHAP|', ascending=False).reset_index(drop=True))
print(shap_rank.head(10).to_string(index=False, float_format=lambda v: f"{v:.4f}"))
plt.figure(figsize=(9, 5))
sns.barplot(data=shap_rank.head(10), x='Mean |SHAP|', y='Feature', color='#3b82f6')
plt.title("Top 10 Drivers of Dropout Risk (Mean |SHAP|)", fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

print("[3/3] LIME explanation for one correctly flagged dropout (median risk among true pos
explainer_lime = lime.lime_tabular.LimeTabularExplainer(
    training_data=X_train.values,
    feature_names=X_train.columns.tolist(),
    class_names=['Retained', 'Dropout'],
    mode='classification',
    random_state=42
)
predict_fn = lambda arr: model.predict_proba(pd.DataFrame(arr, columns=X_train.columns))
tp_idx = np.where((y_pred == 1) & (y_test_arr == 1))[0]
lime_pos = tp_idx[np.argsort(y_prob[tp_idx])[len(tp_idx) // 2]]
exp = explainer_lime.explain_instance(X_test.iloc[lime_pos].values, predict_fn, num_feature
lime_list = exp.as_list()
print(f"Student index {X_test.index[lime_pos]} | predicted dropout risk = {y_prob[lime_pos]}
for feature, weight in lime_list:
    print(f" {feature:<55}: contribution = {weight:+.4f}")
exp.as_pyplot_figure()
plt.title(f"LIME Local Explanation - Student {X_test.index[lime_pos]}", fontsize=12, fontwe
plt.tight_layout()
plt.show()

# Takeaways computed from the results
lime_top = [f for f in X_shap.columns if any(f in name for name, _ in lime_list[:3])]
lime_in_shap = [f for f in lime_top if f in shap_rank['Feature'].head(10).tolist()]

```

```
def direction(feature):
    j = list(X_shap.columns).index(feature)
    r = np.corrcoef(X_shap[feature].values.astype(float), shap_dropout[:, j])[0, 1]
    return 'higher values push risk UP' if r > 0 else 'higher values push risk DOWN'

top10 = shap_rank['Feature'].head(10).tolist()
top3_txt = '; '.join(f"{f}" ({direction(f)})" for f in top10[:3])
sem2_top = [f for f in top10 if f in sem2_cols]
eng_top = [f for f in top10 if f in ENGINEERED]
display(Markdown(f"""
**Step 5b takeaways (computed from the results)**

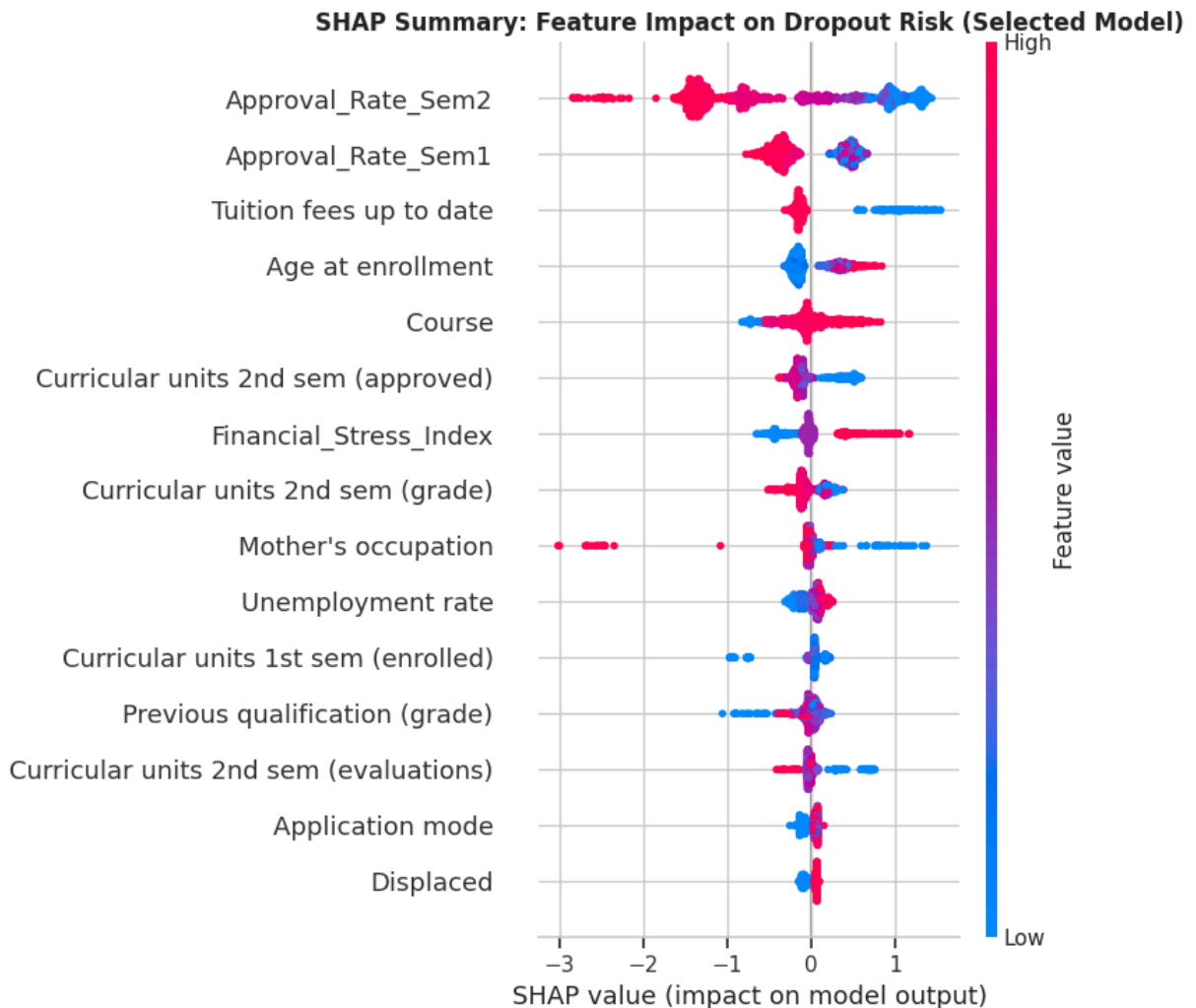
- **Global drivers:** the three largest mean |SHAP| features are {top3_txt}.
- **Timing:** {len(sem2_top)} of the top 10 drivers are Semester 2 variables{' (' + ', '.jo
- **Engineered features:** {len(eng_top)} of the top 10 are features built in Step 3{' (' +
- **Local explanation (LIME):** for student {X_test.index[lime_pos]} (predicted risk {y_pro
- **Sensitive attributes:** 'Gender' and 'Marital status' are not model inputs, so they can
"""))
```

=====

EXPLAINABILITY: SHAP GLOBAL IMPORTANCE & LIME LOCAL EXPLANATION

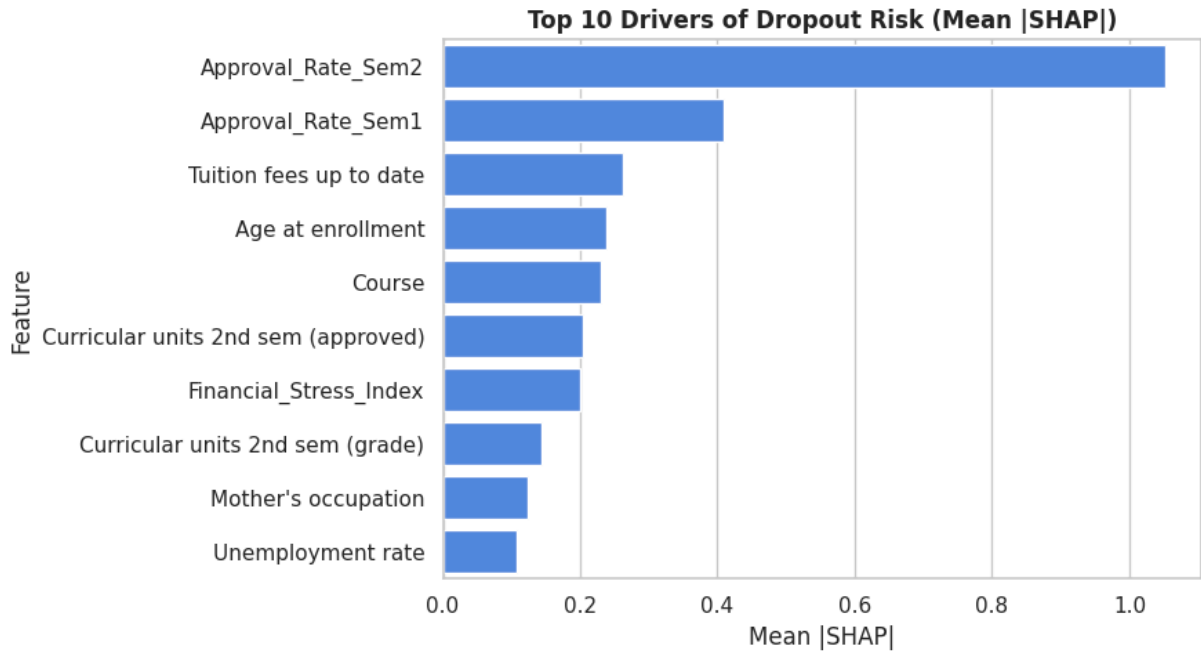
=====

[1/3] SHAP beeswarm: each dot is one student; SHAP values are in log-odds of dropout (n = 885 students)



[2/3] Mean |SHAP| ranking

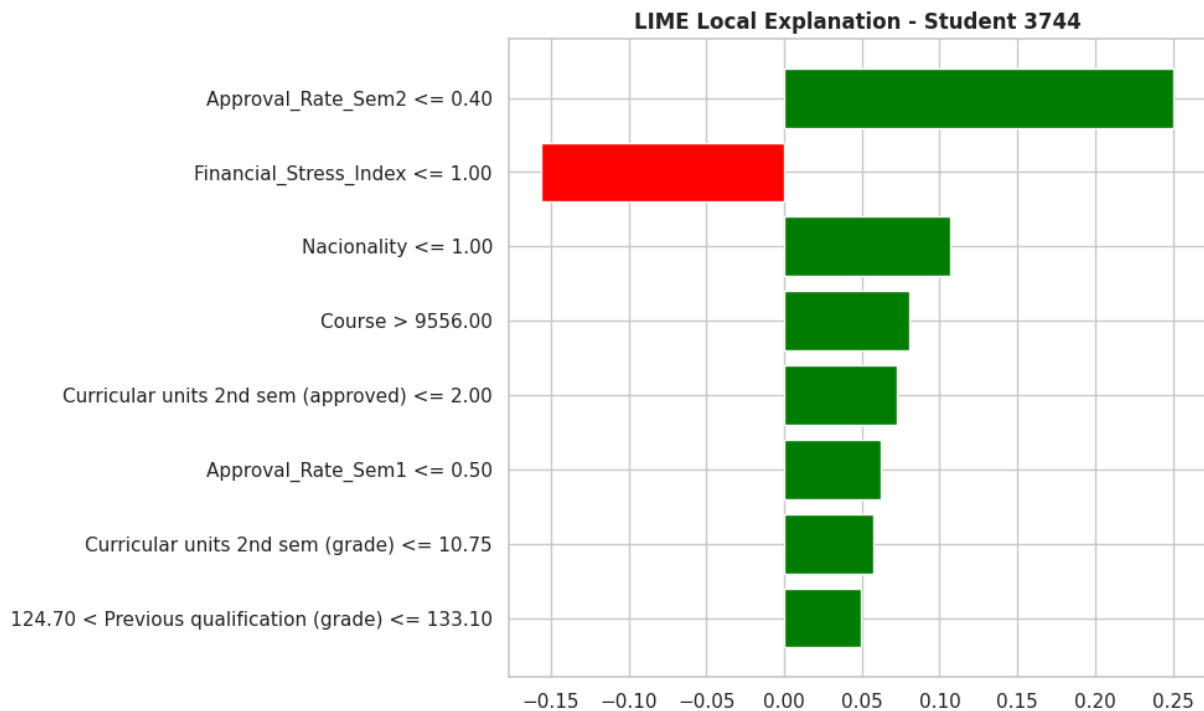
Feature	Mean SHAP
Approval_Rate_Sem2	1.0522
Approval_Rate_Sem1	0.4089
Tuition fees up to date	0.2622
Age at enrollment	0.2375
Course	0.2301
Curricular units 2nd sem (approved)	0.2032
Financial_Stress_Index	0.1992
Curricular units 2nd sem (grade)	0.1437
Mother's occupation	0.1229
Unemployment rate	0.1075



[3/3] LIME explanation for one correctly flagged dropout (median risk among true positives)

Student index 3744 | predicted dropout risk = 97.11%

Approval_Rate_Sem2 <= 0.40	: contribution = +0.2507
Financial_Stress_Index <= 1.00	: contribution = -0.1565
Nacionality <= 1.00	: contribution = +0.1065
Course > 9556.00	: contribution = +0.0806
Curricular units 2nd sem (approved) <= 2.00	: contribution = +0.0724
Approval_Rate_Sem1 <= 0.50	: contribution = +0.0625
Curricular units 2nd sem (grade) <= 10.75	: contribution = +0.0575
124.70 < Previous qualification (grade) <= 133.10	: contribution = +0.0498



Step 5b takeaways (computed from the results)

- Global drivers: the three largest mean |SHAP| features are `Approval_Rate_Sem2` (higher values push risk DOWN); `Approval_Rate_Sem1` (higher values push risk DOWN); `Tuition fees up to date` (higher values push risk DOWN).
- Timing: 3 of the top 10 drivers are Semester 2 variables (`Approval_Rate_Sem2` , `Curricular units 2nd sem (approved)` , `Curricular units 2nd sem (grade)`); the end-of-Semester-2 decision point is what exposes this academic signal to the model.
- Engineered features: 3 of the top 10 are features built in Step 3 (`Approval_Rate_Sem2` , `Approval_Rate_Sem1` , `Financial_Stress_Index`).
- Local explanation (LIME): for student 3744 (predicted risk 97%), the strongest local contributions are `Approval_Rate_Sem2 <= 0.40` (+0.251); `Financial_Stress_Index <= 1.00` (-0.156); `Nacionality <= 1.00` (+0.107). Agreement with the global view: 2 of the 3 LIME features matched to column names also rank in the SHAP top 10, so the two methods only partly agree and the local story should be read with care.
- Sensitive attributes: `Gender` and `Marital status` are not model inputs, so they cannot appear in these attributions.

Step 5c: Does the Model Treat Groups Fairly? (Bias & Fairness Audit)

- Applied Concept: Subgroup Auditing with Demographic-Parity, Equal-Opportunity and Equalized-Odds Metrics.
- How It Works:

- Groups audited: gender, marital status (single vs not), scholarship status (a socio-economic proxy), debtor status and age group (≤ 20 , 21-25, > 25). The first two come from the audit table because they are not model inputs. Groups with fewer than 30 students are excluded from ratios.
- Metrics per group: selection rate, recall (TPR), false-positive rate (FPR) and precision.
- Fairness criteria:
 - *Disparate impact* = $\min_g SR_g / \max_g SR_g$, where SR_g is the share of group g flagged; values below 0.80 are flagged.
 - *Equal opportunity* = $\max_g TPR_g - \min_g TPR_g$; gaps above 0.10 are flagged.
 - *Equalized odds* requires both the TPR and FPR gaps to be small.
- Primary criterion: recall parity. Selection-rate gaps partly reflect real differences in base dropout rates, whereas unequal recall means one group's at-risk students are missed more often, which is the harm that matters here.
- Does excluding sensitive attributes help? A counterfactual model that *is* given gender and marital status is trained and compared with the deployed model, showing both the fairness effect and the accuracy cost of the exclusion.

```
In [6]: # Step 5c: Bias & Fairness Audit (selected model, threshold = 0.50)
print("=" * 80)
print("BIAS & FAIRNESS AUDIT: performance and error rates by sensitive / socio-economic gro
print("=" * 80)

def build_groups(A):
    """Audit groups from the audit table. UCI coding: Gender 1 = male, 0 = female; Marital
    g = pd.DataFrame(index=A.index)
    g['Gender'] = A['Gender'].map({0: 'Female', 1: 'Male'})
    g['Marital status'] = np.where(A['Marital status'] == 1, 'Single', 'Not single')
    g['Scholarship holder'] = A['Scholarship holder'].map({0: 'No scholarship', 1: 'Scholar
    g['Debtor'] = A['Debtor'].map({0: 'Not debtor', 1: 'Debtor'})
    g['Age group'] = pd.cut(A['Age at enrollment'], bins=[0, 20, 25, 200], labels=['<=20',
    return g.reset_index(drop=True)

def group_metrics(groups, y_true, y_hat):
    rows = []
    for attr in groups.columns:
        for level in sorted(groups[attr].dropna().unique()):
            mask = (groups[attr] == level).values
            yt, yp = y_true[mask], y_hat[mask]
            tn_, fp_, fn_, tp_ = confusion_matrix(yt, yp, labels=[0, 1]).ravel()
            rows.append({
                'Attribute': attr, 'Group': level, 'N': int(mask.sum()),
                'Dropout rate': yt.mean(), 'Selection rate': yp.mean(),
                'Recall (TPR)': tp_ / (tp_ + fn_) if (tp_ + fn_) else np.nan,
                'FPR': fp_ / (fp_ + tn_) if (fp_ + tn_) else np.nan,
                'Precision': tp_ / (tp_ + fp_) if (tp_ + fp_) else np.nan,
            })
    return pd.DataFrame(rows)

MIN_GROUP_N = 30 # groups smaller than this are too small for stable rates
```

```

def fairness_summary(gm):
    out = []
    for attr, sub in gm.groupby('Attribute', sort=False):
        sub = sub[sub['N'] >= MIN_GROUP_N]
        if len(sub) < 2:
            continue
        max_sel = sub['Selection rate'].max()
        tpr_gap = sub['Recall (TPR)'].max() - sub['Recall (TPR)'].min()
        fpr_gap = sub['FPR'].max() - sub['FPR'].min()
        out.append({
            'Attribute': attr,
            'Disparate impact': sub['Selection rate'].min() / max_sel if max_sel > 0 else n
            'TPR gap': tpr_gap, 'FPR gap': fpr_gap,
            'Equalized-odds gap': max(tpr_gap, fpr_gap),
        })
    return pd.DataFrame(out)

groups_test = build_groups(audit_test)
gm_base = group_metrics(groups_test, y_test_arr, y_pred)
fair_base = fairness_summary(gm_base)

print("Per-group performance:")
print(gm_base.to_string(index=False, float_format=lambda v: f"{v:.3f}"))
print("\nFairness metrics per attribute (groups with N >= 30):")
print(fair_base.to_string(index=False, float_format=lambda v: f"{v:.3f}"))

flags = []
for _, r in fair_base.iterrows():
    reasons = []
    if r['Disparate impact'] < 0.80:
        reasons.append(f"disparate impact {r['Disparate impact']:.2f} < 0.80")
    if r['TPR gap'] > 0.10:
        reasons.append(f"recall gap {r['TPR gap']:.2f} > 0.10")
    if reasons:
        flags.append((r['Attribute'], ';' .join(reasons)))
    print(f" {'FLAG' if reasons else 'OK' } [{r['Attribute']}] : " + (';' .join(reasons) if

# Does excluding the sensitive attributes matter? Counterfactual model that DOES receive th
X_tr_s = pd.concat([X_train, audit_train[SENSITIVE_COLS]], axis=1)
X_te_s = pd.concat([X_test, audit_test[SENSITIVE_COLS]], axis=1)
model_with = clone(model).fit(X_tr_s, y_train)
prob_with = model_with.predict_proba(X_te_s)[: , 1]
pred_with = (prob_with >= THRESHOLD).astype(int)
fair_with = fairness_summary(group_metrics(groups_test, y_test_arr, pred_with)).set_index('
fair_wo = fair_base.set_index('Attribute')

def gap(fs, attr, col):
    return fs.loc[attr, col] if attr in fs.index else np.nan

unaware_df = pd.DataFrame([
    {'Model': 'Deployed: sensitive attributes excluded', 'Recall': base_metrics['Recall'],
    'Gender TPR gap': gap(fair_wo, 'Gender', 'TPR gap'), 'Gender disparate impact': gap(fa
    'Marital TPR gap': gap(fair_wo, 'Marital status', 'TPR gap')},
    {'Model': 'Counterfactual: Gender + Marital status included', 'Recall': recall_score(y_
    'ROC-AUC': roc_auc_score(y_test_arr, prob_with),

```

```

    'Gender TPR gap': gap(fair_with, 'Gender', 'TPR gap'), 'Gender disparate impact': gap(
    'Marital TPR gap': gap(fair_with, 'Marital status', 'TPR gap')},
])
print("\nEffect of excluding the sensitive attributes (fairness through unawareness check):
print(unaware_df.to_string(index=False, float_format=lambda v: f"{v:.3f}"))

# Visual: recall and selection rate by group
plot_df = gm_base[gm_base['N'] >= MIN_GROUP_N].copy()
plot_df['Label'] = plot_df['Attribute'] + ': ' + plot_df['Group']
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
sns.barplot(data=plot_df, y='Label', x='Recall (TPR)', ax=axes[0], color='#3b82f6')
axes[0].axvline(TARGET_RECALL, color='red', ls='--', label=f'Target {TARGET_RECALL}')
axes[0].set_title('Recall by Group (equal opportunity)')
axes[0].legend()
sns.barplot(data=plot_df, y='Label', x='Selection rate', ax=axes[1], color='#f59e0b')
axes[1].set_title('Share of Students Flagged by Group (demographic parity)')
plt.tight_layout()
plt.show()

# Takeaways computed from the results
worst = fair_base.sort_values('TPR gap', ascending=False).iloc[0]
g_gender = unaware_df.iloc[0]['Gender TPR gap'], unaware_df.iloc[1]['Gender TPR gap']
display(Markdown(f"""
**Step 5c takeaways (computed from the results)**

- **Audit scope:** {len(fair_base)} attributes (gender, marital status, scholarship, debtor
- **Flags:** {'; '.join(f'**{a}** ({r})' for a, r in flags) if flags else 'no attribute bre
- **Largest recall gap:** **{worst['Attribute']}** at {worst['TPR gap']:.3f} (disparate imp
- **How to read disparate impact:** selection-rate ratios partly mirror genuine differences
- **Excluding sensitive attributes:** the gender recall gap is {g_gender[0]:.3f} without th
"""))

```

=====

BIAS & FAIRNESS AUDIT: performance and error rates by sensitive / socio-economic group

=====

Per-group performance:

	Attribute	Group	N	Dropout rate	Selection rate	Recall (TPR)	FPR	Precision
0.648	Gender	Female	564	0.245	0.342	0.906	0.160	
0.724	Gender	Male	321	0.455	0.576	0.918	0.291	
0.797	Marital status	Not single	110	0.527	0.627	0.948	0.269	
0.660	Marital status	Single	775	0.292	0.399	0.903	0.191	
0.694	Scholarship holder	No scholarship	654	0.390	0.520	0.925	0.261	
0.605	Scholarship holder	Scholarship	231	0.126	0.165	0.793	0.074	
0.750	Debtor	Debtor	90	0.578	0.756	0.981	0.447	
0.671	Debtor	Not debtor	795	0.292	0.390	0.897	0.181	
0.646	Age group	21-25	157	0.357	0.522	0.946	0.287	
0.554	Age group	<=20	513	0.199	0.288	0.804	0.161	
0.838	Age group	>25	215	0.586	0.688	0.984	0.270	

Fairness metrics per attribute (groups with N >= 30):

	Attribute	Disparate impact	TPR gap	FPR gap	Equalized-odds gap
	Gender	0.594	0.012	0.132	0.132
	Marital status	0.636	0.046	0.078	0.078
	Scholarship holder	0.316	0.132	0.186	0.186
	Debtor	0.516	0.084	0.266	0.266
	Age group	0.419	0.180	0.127	0.180

FLAG [Gender]: disparate impact 0.59 < 0.80

FLAG [Marital status]: disparate impact 0.64 < 0.80

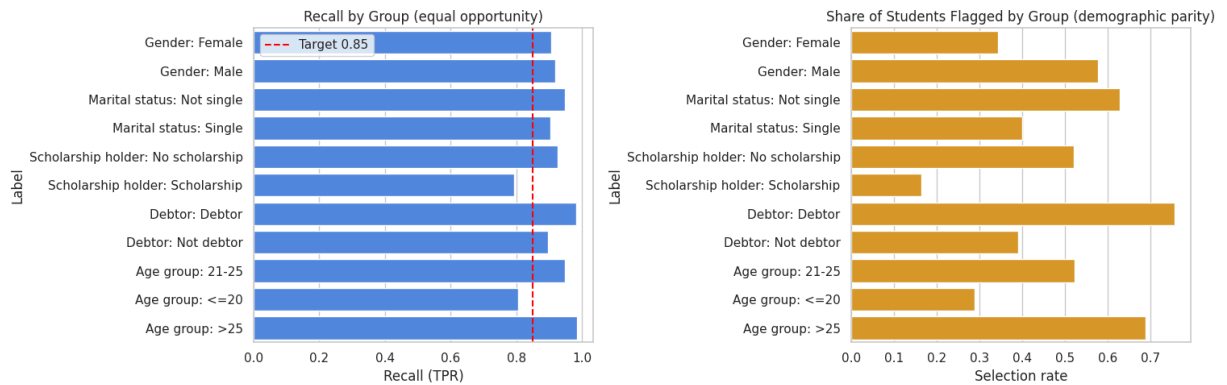
FLAG [Scholarship holder]: disparate impact 0.32 < 0.80; recall gap 0.13 > 0.10

FLAG [Debtor]: disparate impact 0.52 < 0.80

FLAG [Age group]: disparate impact 0.42 < 0.80; recall gap 0.18 > 0.10

Effect of excluding the sensitive attributes (fairness through unawareness check):

	Model	Recall	ROC-AUC	Gender TPR gap	Gender di
Disparate impact	Marital TPR gap				
0.594	Deployed: sensitive attributes excluded	0.912	0.937	0.012	
					0.046
0.560	Counterfactual: Gender + Marital status included	0.905	0.937	0.041	
					0.011



Step 5c takeaways (computed from the results)

- Audit scope: 5 attributes (gender, marital status, scholarship, debtor status, age group); groups with fewer than 30 students are excluded because their rates are unstable.
- Flags: Gender (disparate impact $0.59 < 0.80$); Marital status (disparate impact $0.64 < 0.80$); Scholarship holder (disparate impact $0.32 < 0.80$; recall gap $0.13 > 0.10$); Debtor (disparate impact $0.52 < 0.80$); Age group (disparate impact $0.42 < 0.80$; recall gap $0.18 > 0.10$).
- Largest recall gap: Age group at 0.180 (disparate impact 0.419).
- How to read disparate impact: selection-rate ratios partly mirror genuine differences in dropout rate between groups (see the **Dropout rate** column), so recall parity is the primary criterion; the harm to avoid is a group of at-risk students being missed more often than others.
- Excluding sensitive attributes: the gender recall gap is 0.012 without them versus 0.041 when they are included, and ROC-AUC moves from 0.937 to 0.937. Excluding them costs little accuracy; because other features can act as proxies, outcomes are still audited by group rather than assumed fair.

Step 5d: Can the Gaps Be Reduced? (Bias Mitigation, Evaluated)

- Applied Concept: In-Processing and Post-Processing Mitigation with a Pre-Declared Decision Rule.
- How It Works:
 - Target: the attribute with the largest baseline recall gap in Step 5c.
 - Reweighting (in-processing): each training row receives weight $w(g, y) = \frac{P(g) P(y)}{P(g, y)}$, which makes group and outcome statistically independent in the weighted training set; the model is then refitted.
 - Group thresholds (post-processing): a separate decision threshold per group is chosen on out-of-fold training predictions so that each group's recall matches the overall level; the test set is never consulted when choosing thresholds.

- Evaluation: both variants are scored on the untouched hold-out set against the baseline: overall recall, precision, F_2 , ROC-AUC, and the group recall and false-positive gaps.
- Decision rule (fixed before looking at results): adopt a mitigation only if it narrows the recall gap and keeps overall recall at or above 0.85 (if the baseline already misses 0.85, recall must not fall by more than 0.02 from the baseline); otherwise keep the baseline and report the residual gap as a limitation.

```
In [7]: # Step 5d: Bias Mitigation - implemented and evaluated on the held-out set
from sklearn.pipeline import Pipeline

def fit_weighted(est, X, y, sample_weight):
    """Pass sample weights to the final step when the estimator is a Pipeline."""
    if isinstance(est, Pipeline):
        return est.fit(X, y, **{f"{est.steps[-1][0]}__sample_weight": sample_weight})
    return est.fit(X, y, sample_weight=sample_weight)

print("=" * 80)
print("BIAS MITIGATION: REWEIGHING (in-processing) vs GROUP THRESHOLDS (post-processing)")
print("=" * 80)
focus_attr = fair_base.sort_values('TPR gap', ascending=False).iloc[0]['Attribute']
print(f"Attribute with the largest recall gap (mitigation target): {focus_attr}")

groups_train = build_groups(audit_train)
g_tr = groups_train[focus_attr].astype(str).values
g_te = groups_test[focus_attr].astype(str).values

# 1. Reweighting (Kamiran & Calders):  $w(g, y) = P(g) * P(y) / P(g, y)$ , learned on training data
w = np.ones(len(y_train_arr))
for gv in np.unique(g_tr):
    for yv in (0, 1):
        cell = (g_tr == gv) & (y_train_arr == yv)
        if cell.any():
            w[cell] = (g_tr == gv).mean() * (y_train_arr == yv).mean() / cell.mean()
model_rw = fit_weighted(clone(model), X_train, y_train, w)
prob_rw = model_rw.predict_proba(X_test)[: , 1]
pred_rw = (prob_rw >= THRESHOLD).astype(int)

# 2. Group-specific thresholds chosen on OUT-OF-FOLD training predictions (the test set is
oof = cross_val_predict(clone(model), X_train, y_train, cv=cv, method='predict_proba')[: , 1]
target_tpr = recall_score(y_train_arr, (oof >= THRESHOLD).astype(int))
grid = np.linspace(0.05, 0.95, 91)
group_thr = {}
for gv in np.unique(g_tr):
    m_ = g_tr == gv
    ok = [t for t in grid if recall_score(y_train_arr[m_], (oof[m_] >= t).astype(int), zero
group_thr[gv] = max(ok) if ok else grid.min()
thr_vec = np.array([group_thr.get(gv, THRESHOLD) for gv in g_te])
pred_thr = (y_prob >= thr_vec).astype(int)
print(f"Group thresholds (equalising out-of-fold recall to the overall level {target_tpr:.3
{k: round(float(v), 2) for k, v in group_thr.items()})

def evaluate_variant(name, y_hat, prob):
```

```

fs = fairness_summary(group_metrics(groups_test[[focus_attr]], y_test_arr, y_hat))
row = fs.iloc[0] if len(fs) else None
return {
    'Variant': name,
    'Recall': recall_score(y_test_arr, y_hat),
    'Precision': precision_score(y_test_arr, y_hat, zero_division=0),
    'F2-Score': fbeta_score(y_test_arr, y_hat, beta=2),
    'ROC-AUC': roc_auc_score(y_test_arr, prob),
    f'Disparate impact ({focus_attr})': row['Disparate impact'] if row is not None else
    f'TPR gap ({focus_attr})': row['TPR gap'] if row is not None else np.nan,
    f'FPR gap ({focus_attr})': row['FPR gap'] if row is not None else np.nan,
}

mitigation_results = pd.DataFrame([
    evaluate_variant('Baseline (selected model)', y_pred, y_prob),
    evaluate_variant('Reweighting (in-processing)', pred_rw, prob_rw),
    evaluate_variant('Group thresholds (post-processing)', pred_thr, y_prob),
])
print("\nMitigation comparison on the held-out test set:")
print(mitigation_results.to_string(index=False, float_format=lambda v: f"{v:.3f}"))

# 3. Decision rule: adopt a mitigation only if it narrows the recall gap while overall reca
gap_col = f'TPR gap ({focus_attr})'
base_gap = mitigation_results.loc[0, gap_col]
candidates = mitigation_results.iloc[1:]
base_recall = mitigation_results.loc[0, 'Recall']
recall_floor = TARGET_RECALL if base_recall >= TARGET_RECALL else base_recall - 0.02 # if
eligible = candidates[(candidates['Recall'] >= recall_floor) & (candidates[gap_col] < base_
final_decision = eligible.sort_values(gap_col).iloc[0]['Variant'] if len(eligible) else 'Ba
print(f"\nDecision rule -> adopt: {final_decision}")

fig, ax = plt.subplots(figsize=(9, 4))
sns.barplot(data=mitigation_results, y='Variant', x=gap_col, color='#10b981', ax=ax)
ax.set_title(f'Recall Gap Across {focus_attr} Groups, Before and After Mitigation')
plt.tight_layout()
plt.show()

# Takeaways computed from the results
rows = []
for i in (1, 2):
    r = mitigation_results.loc[i]
    rows.append(f"- **{r['Variant']}**: recall gap {base_gap:.3f} -> {r[gap_col]:.3f}; over
display(Markdown(f"""
**Step 5d takeaways (computed from the results)**

- **Mitigation target:** `{focus_attr}`, the attribute with the largest baseline recall gap
{chr(10)}.join(rows))
- **Decision:** **{final_decision}**. {f'The adopted variant narrows the gap while keeping
- **Trade-off:** fairness interventions move the operating point; the precision change abov
"""))

```

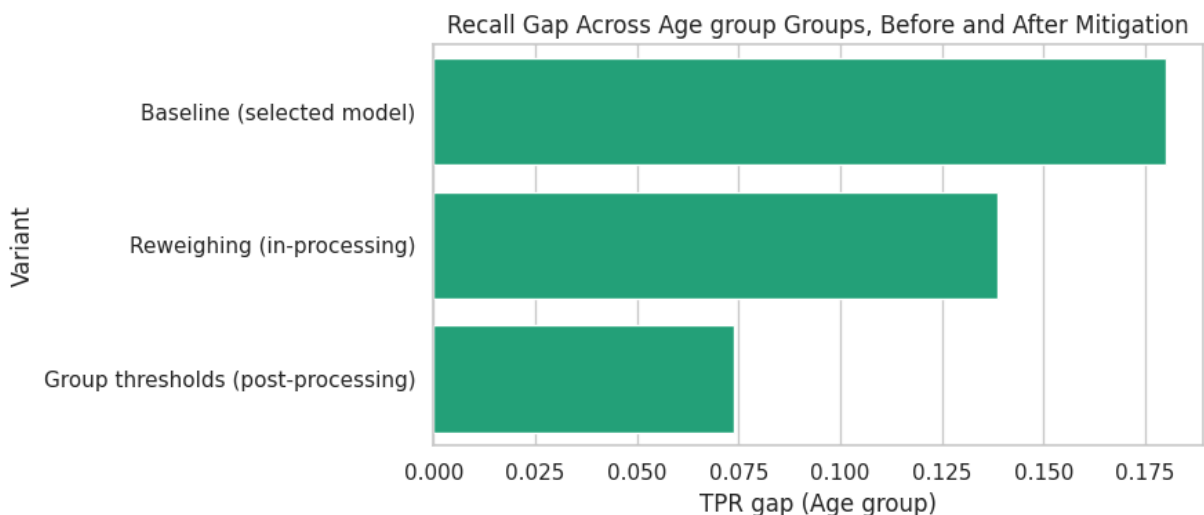
BIAS MITIGATION: REWEIGHING (in-processing) vs GROUP THRESHOLDS (post-processing)

Attribute with the largest recall gap (mitigation target): Age group
 Group thresholds (equalising out-of-fold recall to the overall level 0.880): {'21-25': 0.53, '<=20': 0.3, '>25': 0.79}

Mitigation comparison on the held-out test set:

	Variant	Recall	Precision	F2-Score	ROC-AUC	Disparate impact
(Age group)	TPR gap (Age group)	FPR gap (Age group)				
	Baseline (selected model)	0.912	0.685	0.855	0.937	
0.419	0.180	0.127				
	Reweighting (in-processing)	0.901	0.642	0.834	0.935	
0.552	0.139	0.127				
	Group thresholds (post-processing)	0.898	0.614	0.822	0.937	
0.741	0.074	0.214				

Decision rule -> adopt: Group thresholds (post-processing)



Step 5d takeaways (computed from the results)

- Mitigation target: Age group , the attribute with the largest baseline recall gap (0.180).
- Reweighting (in-processing): recall gap 0.180 -> 0.139; overall recall 0.912 -> 0.901; precision 0.685 -> 0.642.
- Group thresholds (post-processing): recall gap 0.180 -> 0.074; overall recall 0.912 -> 0.898; precision 0.685 -> 0.614.
- Decision: Group thresholds (post-processing). The adopted variant narrows the gap while keeping recall at or above 0.850.
- Trade-off: fairness interventions move the operating point; the precision change above is the extra advisory workload that equal-opportunity buys.

Step 5e: Findings, Limitations & Recommendations

- Applied Concept: Synthesis of Evidence for a Technical Audience.

- How It Works: the next cell assembles the headline results, the limitations that qualify them, and operating recommendations directly from the variables computed in Steps 4 and 5a-5d, so the summary cannot drift from the evidence.

```
In [8]: # Step 5e: Findings, Limitations & Recommendations (assembled from the computed results above)
best = summary_df.iloc[0]
adopted = final_decision
adopted_row = mitigation_results[mitigation_results['Variant'] == adopted].iloc[0]
display(Markdown(f"""
### Findings

| Question | Answer (from this run) |
|---|---|
| Selected model | **{best_model_name}** (CV F2 {best['Best CV F2-Score']:.3f}) |
| Hold-out recall / precision | **{base_metrics['Recall']:.3f}** / **{base_metrics['Precision']:.3f}** |
| Recall target ({TARGET_RECALL}) | **{'Met' if base_metrics['Recall'] >= TARGET_RECALL else 'Not Met'} |
| ROC-AUC / PR-AUC | {base_metrics['ROC-AUC']:.3f} / {base_metrics['PR-AUC']:.3f} |
| Top drivers (SHAP) | {'', '.join('' + f + '' for f in shap_rank['Feature'].head(3))} |
| Gain from Semester 2 data | ROC-AUC {auc_no_sem2:.3f} -> {base_metrics['ROC-AUC']:.3f} |
| Largest fairness gap | {focus_attr}: recall gap {base_gap:.3f} |
| Adopted fairness treatment | **{adopted}** (recall {adopted_row['Recall']:.3f}, precision {adopted_row['Precision']:.3f})

### Limitations

1. **Class imbalance:** dropouts are {y_train_arr.mean():.1%} of the data; weighting raises
2. **Label censoring:** students still *Enrolled* are labelled Retained although their final
3. **Generalisability:** one institution, one country, one cohort period; performance must
4. **Proxies:** excluding gender and marital status does not remove correlated information,
5. **Small groups:** subgroups below {MIN_GROUP_N} students are excluded from fairness ratio
6. **Unobserved causes:** illness, family and work changes are invisible to administrative
7. **Model selection margin:** the top models are close in CV F2, so the choice among them

### Recommendations

- **Operate** the end-of-Semester-2 scoring as advisor decision support, not automated action
- **Monitor** recall, flag rate and the per-group recall gap each cycle, and retrain when data changes
- **Route** flagged students to the interventions below, matching the support to the dominant risk profile

| Risk profile | Recommended intervention |
|---|---|
| High risk and tuition not up to date / debtor | Financial-aid and emergency-grant review
| High risk and low Semester 1-2 approval rate | Peer tutoring and academic advising |
| High risk and mature student (> 25) | Non-traditional-student support and work-study mentorship
| Moderate risk (probability 0.4-0.6) | Automated check-in and counselling survey |
"""))
```

Findings

Question	Answer (from this run)
Selected model	XGBoost (CV F2 0.825)
Hold-out recall / precision	0.912 / 0.685 at threshold 0.5
Recall target (0.85)	Met
ROC-AUC / PR-AUC	0.937 / 0.911
Top drivers (SHAP)	Approval_Rate_Sem2 , Approval_Rate_Sem1 , Tuition fees up to date
Gain from Semester 2 data	ROC-AUC 0.912 -> 0.937
Largest fairness gap	Age group: recall gap 0.180
Adopted fairness treatment	Group thresholds (post-processing) (recall 0.898, precision 0.614, residual gap 0.074)

Limitations

1. Class imbalance: dropouts are 32.1% of the data; weighting raises recall at the cost of precision (extra false alarms).
2. Label censoring: students still *Enrolled* are labelled Retained although their final outcome is unknown.
3. Generalisability: one institution, one country, one cohort period; performance must be re-validated before use elsewhere.
4. Proxies: excluding gender and marital status does not remove correlated information, so group-level audits must continue in production.
5. Small groups: subgroups below 30 students are excluded from fairness ratios; their treatment cannot be certified from this data.
6. Unobserved causes: illness, family and work changes are invisible to administrative records.
7. Model selection margin: the top models are close in CV F2, so the choice among them is partly a judgement call.

Recommendations

- Operate the end-of-Semester-2 scoring as advisor decision support, not automated action; every flag triggers a human check-in.
- Monitor recall, flag rate and the per-group recall gap each cycle, and retrain when drift or a gap above 0.10 appears.

- Route flagged students to the interventions below, matching the support to the dominant risk signal.

Risk profile	Recommended intervention
High risk and tuition not up to date / debtor	Financial-aid and emergency-grant review
High risk and low Semester 1-2 approval rate	Peer tutoring and academic advising
High risk and mature student (> 25)	Non-traditional-student support and work-study mentorship
Moderate risk (probability 0.4-0.6)	Automated check-in and counselling survey

Step 6: Final Presentation & Communication

- Applied Concept: Dual-Audience Communication.
- Technical deck (this notebook): the export cell below runs `nbconvert` on the saved notebook and produces an HTML report, an interactive slide deck, a LaTeX file and a PDF. A new slide starts at each section-header markdown cell, and each slide shows that cell's reasoning together with the code and outputs beneath it, so the deck reads as one argument: framing, data, features, models, explanation, fairness, mitigation, findings, reproducibility. This notebook therefore yields 12 slides.
- Executive deck (separate PowerPoint): written for university leadership and academic advisors, it translates the measured results into business terms: recall and precision as students caught versus advisory workload, projected tuition revenue retained against intervention cost, the fairness audit and its mitigation, and a phased roll-out. Every figure on it is taken from the Step 4 and Step 5 takeaway blocks, and it targets 12 slides or fewer.
- To regenerate the technical deck: run all cells, save the notebook (`Ctrl+S`) so the file on Drive contains the outputs, then run the export cell.

```
In [9]: import os
import json
import glob
import subprocess
import sys
import asyncio
from google.colab import drive, output

# 0. Configure nest_asyncio to allow asyncio execution inside Colab's active loop
try:
    import nest_asyncio
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "nest_asyncio", "-q"])
```

```

import nest_asyncio

nest_asyncio.apply()

# Ensure BeautifulSoup4 is available for HTML cell parsing
try:
    from bs4 import BeautifulSoup
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "beautifulsoup4", "-q"])
    from bs4 import BeautifulSoup

# Install Async Playwright and Chromium browser binary for pixel-perfect PDF rendering
try:
    from playwright.async_api import async_playwright
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "playwright", "-q"])
    subprocess.check_call([sys.executable, "-m", "playwright", "install", "chromium", "--wi
    from playwright.async_api import async_playwright

# 1. Mount Google Drive
drive.mount('/content/drive', force_remount=False)

drive_folder = '/content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT'
os.makedirs(drive_folder, exist_ok=True)

# 2. Resolve Active Notebook Filename dynamically
notebook_filename = None

# Attempt 1: Google Colab Frontend JavaScript Evaluation
try:
    js_name = output.eval_js('document.querySelector("#doc-name") ? document.querySelector(
    if js_name and isinstance(js_name, str):
        js_name = js_name.strip()
        if js_name:
            notebook_filename = js_name if js_name.endswith('.ipynb') else f"{js_name}.ipyn
except Exception:
    pass

# Attempt 2: Fallback to the most recently modified .ipynb file in the directory
if not notebook_filename:
    pattern = os.path.join(drive_folder, "*.ipynb")
    notebooks = [f for f in glob.glob(pattern) if not os.path.basename(f).startswith('.ipyn
    if notebooks:
        notebooks.sort(key=os.path.getmtime, reverse=True)
        notebook_filename = os.path.basename(notebooks[0])

# Fallback default if directory has no notebooks
if not notebook_filename:
    notebook_filename = 'Capstone05.ipynb'

notebook_path = os.path.join(drive_folder, notebook_filename)
base_name = os.path.splitext(notebook_filename)[0]

# File Naming Convention: "Technical Slides_<NotebookName>"
output_prefix = f"Technical Slides_{base_name}"

```

```

interactive_slides_filename = f"{output_prefix}.slides.html"
html_filename = f"{output_prefix}.html"
latex_filename = f"{output_prefix}.tex"
pdf_filename = f"{output_prefix}.pdf"

interactive_slides_path = os.path.join(drive_folder, interactive_slides_filename)
html_output_path = os.path.join(drive_folder, html_filename)
latex_output_path = os.path.join(drive_folder, latex_filename)
pdf_output_path = os.path.join(drive_folder, pdf_filename)

print(f"Target Active Notebook: {notebook_path}")
print(f"Output Prefix: {output_prefix}")

try:
    # Read notebook structure for slide splitting bounds
    with open(notebook_path, 'r', encoding='utf-8') as f:
        nb_data = json.load(f)

    raw_cells = nb_data.get('cells', [])

    # STEP A: Export Plain HTML via nbconvert FIRST so we can use its styled output
    print("Converting to HTML document...")
    subprocess.run([
        "jupyter", "nbconvert", "--to", "html",
        "--output", html_filename,
        "--output-dir", drive_folder,
        notebook_path
    ], check=True)
    print(f"✓ Exported Plain HTML: {html_output_path}")

    # STEP B: Parse rendered HTML to extract Jupyter CSS & top-level DOM cells
    print("Generating Styled Interactive Slides from rendered HTML...")
    with open(html_output_path, "r", encoding="utf-8") as h_file:
        rendered_html_doc = h_file.read()

    soup = BeautifulSoup(rendered_html_doc, 'html.parser')

    # Extract all CSS styles from Jupyter's rendered HTML head
    jupyter_styles = "\n".join([str(style) for style in soup.find_all('style')])

    # Extract ONLY top-level cell containers to avoid matching inner wrapper divs
    top_cells = []
    for div in soup.find_all('div'):
        classes = div.get('class', [])
        if any(c in classes for c in ['jp-Cell', 'cell']):
            # Check if this cell div is nested inside another cell container
            is_nested = div.find_parent(lambda p: p.name == 'div' and any(c in p.get('class')
                if not is_nested:
                    top_cells.append(div)

    # Map raw notebook cells to rendered DOM elements without duplicate splitting
    slides_rendered = []
    current_slide = []

    for idx, cell_json in enumerate(raw_cells):
        source = "".join(cell_json.get('source', []))

```

```

cell_type = cell_json.get('cell_type')

# Check if this cell should trigger a slide break
if cell_type == 'markdown' and any(marker in source for marker in ['# ', '## Step',
    if current_slide:
        slides_rendered.append(current_slide)
        current_slide = []

if idx < len(top_cells):
    current_slide.append(str(top_cells[idx]))

if current_slide:
    slides_rendered.append(current_slide)

# Build Interactive HTML Slides preserving exact Jupyter styling + 200% double height
slides_html_content = f"""<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Technical Presentation - {output_prefix}</title>
    {jupyter_styles}
    <style>
        body, html {{
            margin: 0;
            padding: 0;
            width: 100%;
            height: 100%;
            overflow: hidden;
            background-color: #0f172a;
            font-family: var(--jp-ui-font-family, sans-serif);
        }}
        #deck-container {{
            position: relative;
            width: 100%;
            height: 100%;
            overflow-y: auto;
        }}
        .slide {{
            display: none;
            position: absolute;
            top: 0;
            left: 0;
            width: 100%;
            min-height: 200vh;
            height: 200%;
            box-sizing: border-box;
            padding: 60px 10%;
            background: #ffffff;
            overflow-y: auto;
            color: #1e293b;
        }}
        .slide.active {{
            display: block !important;
        }}
        #nav-toolbar {{
            position: fixed;

```

```

        bottom: 30px;
        right: 30px;
        z-index: 100000;
        display: flex;
        gap: 15px;
        background: rgba(15, 23, 42, 0.95);
        padding: 12px 24px;
        border-radius: 30px;
        align-items: center;
        border: 1px solid #334155;
        box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.3);
    }}
    .nav-btn {{
        background: #3b82f6;
        border: none;
        padding: 8px 18px;
        font-weight: bold;
        border-radius: 20px;
        cursor: pointer;
        color: #ffffff;
    }}
    .nav-btn:hover {{
        background: #2563eb;
    }}
    #progress-text {{
        color: #f1f5f9;
        font-size: 14px;
        font-weight: bold;
    }}
</style>
</head>
<body>
    <div id="deck-container">
        """

for idx, slide_cells in enumerate(slides_rendered):
    active_class = "active" if idx == 0 else ""
    slides_html_content += f'        <div class="slide {active_class}" id="slide-{idx}"'
    for cell_html in slide_cells:
        slides_html_content += f'                {cell_html}\n'
    slides_html_content += f'        </div>\n'

slides_html_content += f'"""
</div>
<div id="nav-toolbar">
    <button class="nav-btn" onclick="changeSlide(-1)">◀ Prev</button>
    <span id="progress-text">Slide 1 of {len(slides_rendered)}</span>
    <button class="nav-btn" onclick="changeSlide(1)">Next ▶</button>
</div>
<script>
    var currentSlideIndex = 0;
    var totalSlides = {len(slides_rendered)};
    function changeSlide(direction) {{
        var currentSlide = document.getElementById("slide-" + currentSlideIndex);
        if (currentSlide) currentSlide.classList.remove("active");
        currentSlideIndex += direction;

```

```

        if (currentSlideIndex < 0) currentSlideIndex = 0;
        if (currentSlideIndex >= totalSlides) currentSlideIndex = totalSlides - 1;
        var activeSlide = document.getElementById("slide-" + currentSlideIndex);
        if (activeSlide) {{
            activeSlide.classList.add("active");
            activeSlide.scrollTop = 0;
        }}
        document.getElementById("progress-text").innerText = "Slide " + (currentSlideIndex + 1)
    }}
    document.addEventListener("keydown", function(e) {{
        if (e.key === "ArrowRight" || e.key === " ") {{
            e.preventDefault(); changeSlide(1);
        }} else if (e.key === "ArrowLeft") {{
            e.preventDefault(); changeSlide(-1);
        }}
    }});
</script>
</body>
</html>
"""

```

Save interactive slides

```

with open(interactive_slides_path, "w", encoding="utf-8") as out_f:
    out_f.write(slides_html_content)
print(f"✓ Exported Dynamic Slides: {interactive_slides_path}")

```

STEP C: Export LaTeX template via nbconvert

```

print("Converting to LaTeX template...")
subprocess.run([
    "jupyter", "nbconvert", "--to", "latex",
    "--output", latex_filename,
    "--output-dir", drive_folder,
    notebook_path
], check=True)
print(f"✓ Exported LaTeX: {latex_output_path}")

```

STEP D: Export PDF via Async Playwright safely within Colab event loop

```

print("Converting HTML to Pixel-Perfect PDF using Headless Chromium...")

```

```

async def generate_pdf_from_html():
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        page = await browser.new_page()
        await page.goto(f"file://{os.path.abspath(html_output_path)}", wait_until="networkidle")
        await page.pdf(
            path=pdf_output_path,
            format="Letter",
            print_background=True,
            margin={"top": "0.5in", "bottom": "0.5in", "left": "0.5in", "right": "0.5in"}
        )
        await browser.close()

```

```

asyncio.get_event_loop().run_until_complete(generate_pdf_from_html())

```

```

print(f"✓ Exported Pixel-Perfect PDF: {pdf_output_path}")
print("\n✓ All file exports successfully processed and completed!")

```

```
except Exception as e:
    print(f"✗ Error occurred during conversion tasks: {e}")
```

Mounted at /content/drive

Target Active Notebook: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/Rex Villamar_Pillar 5 Capstone Project.ipynb

Output Prefix: Technical Slides_Rex Villamar_Pillar 5 Capstone Project

Converting to HTML document...

✓ Exported Plain HTML: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/Technical Slides_Rex Villamar_Pillar 5 Capstone Project.html

Generating Styled Interactive Slides from rendered HTML...

✓ Exported Dynamic Slides: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/Technical Slides_Rex Villamar_Pillar 5 Capstone Project.slides.html

Converting to LaTeX template...

✓ Exported LaTeX: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/Technical Slides_Rex Villamar_Pillar 5 Capstone Project.tex

Converting HTML to Pixel-Perfect PDF using Headless Chromium...

✓ Exported Pixel-Perfect PDF: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/Technical Slides_Rex Villamar_Pillar 5 Capstone Project.pdf

✓ All file exports successfully processed and completed!

Step 7: GitHub Profile & Upload Structure

- Applied Concept: Production Project Structuring and Reproducible Environments.
- How It Works:
 - Folder layout: the first cell creates `src/`, `notebooks/`, `data/raw/`, `data/processed/`, `models/` and `reports/` in the project folder on Drive.
 - Environment and documentation: the second cell writes `requirements.txt` (every library the notebook imports) and a `README.md` covering the project overview, repository structure and quickstart.
 - What gets committed: this notebook under `notebooks/`, the artefacts written in Step 4 (`models/*.joblib`, `model_comparison.csv`, `best_model_config.json`), and the exported decks from Step 6 under `reports/`.

```
In [10]: import os

# Base folder on your Google Drive
base_dir = "/content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT"

# Directories required for the repository layout (Step 7 rubric)
directories = [
    'src',
    'notebooks',
    'data/raw',
    'data/processed',
    'models',
    'reports'
]
```

```

for folder in directories:
    path = os.path.join(base_dir, folder)
    os.makedirs(path, exist_ok=True)
    print(f"Created: {path}")

```

```

Created: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/src
Created: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/notebooks
Created: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/data/raw
Created: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/data/processed
Created: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/models
Created: /content/drive/MyDrive/000_AIM PGD Program/PILLAR 5 CAPSTONE PROJECT/reports

```

```

In [11]: # Step 7: requirements.txt and README.md for the repository
requirements_content = """pandas>=2.0.0
numpy>=1.24.0
scikit-learn>=1.2.0
matplotlib>=3.7.0
seaborn>=0.12.0
shap>=0.41.0
lime>=0.2.0
xgboost>=2.0.0
lightgbm>=4.0.0
joblib>=1.2.0
"""

with open("requirements.txt", "w") as f:
    f.write(requirements_content)

readme_content = """# Early Warning Student Dropout Risk Prediction System

## Project Overview
End-to-end machine learning pipeline that flags students at risk of dropping out at the end

## Repository Structure
```text
|—— data/ # Raw and processed data files
|—— notebooks/ # Full analysis notebook (Steps 1-7)
|—— src/ # Python modules extracted from the notebook
|—— models/ # Tuned models (.joblib), comparison table (.csv), final configur
|—— reports/ # Technical and executive slide decks
|—— requirements.txt # Environment dependencies
|—— README.md # Setup and usage documentation
```

## Quickstart
1. Install dependencies: `pip install -r requirements.txt`
2. Open the notebook in `notebooks/` and run all cells from top to bottom.
"""

with open("README.md", "w") as f:
    f.write(readme_content)

print("✅ requirements.txt and README.md created successfully!")

```

✅ requirements.txt and README.md created successfully!